Handbook thiết kế accelerator YOLOv10n descriptor-driven Đặc tả kiến trúc, register map, định dạng LAYER_DESC/TILE_DESC và sơ đồ block RTL để hiện thực hóa GLB_BANK / ROUTER_CLUSTER / PE_CLUSTER Tác giả: OpenAI GPT-5.4 Pro Ngày biên soạn: 2026-03-07 Ngôn ngữ: Tiếng Việt Kiểu tài liệu: Handbook kỹ thuật / kiến trúc / RTL spec Handbook này hệ thống hóa chuỗi nghiên cứu trước đó và mở rộng thành một bản spec RTL đầy đủ cho accelerator YOLOv10n descriptor-driven. Nội dung tập trung vào: primitive của mô hình, kiến trúc top-level 4 super-cluster, GLB_BANK/ROUTER_CLUSTER/PE_CLUSTER, PPU, register map, LAYER_DESC/TILE_DESC, scheduler, swizzle, decomposition RTL, verify plan và references.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 2 Mục lục

Mục lục 2 1. Mục tiêu, phạm vi và triết lý tài liệu 5 1.1. Bài toán được chốt như thế nào . . . . . 5 1.2. Những giả định nền . . . . . 5 2. Cơ sở nghiên cứu và các tham chiếu phải bám 6 Bảng 2-1. Bộ tham chiếu chính dùng xuyên suốt tài liệu . . . . . 6 3. YOLOv10n nhìn từ góc phần cứng 6 3.1. Primitive thực sự cần support . . . . . 6 Bảng 3-1. Ánh xạ primitive phần cứng cho các module chính của YOLOv10n . . . . . 7 3.2. Các đặc điểm khiến YOLOv10n không thể chạy tối ưu bằng một dataflow duy nhất . . . . . 7 3.3. Những gì không nên làm ở silicon V1 . . . . . 7 4. Mục tiêu phần cứng và triết lý thiết kế 8 Bảng 4-1. Ba trục tối ưu của accelerator . . . . . 8 4.1. Nguyên tắc chuẩn để chọn mode . . . . . 8 5. Kiến trúc top-level của chip 8 Bảng 5-1. Khuyến nghị về các fabric liên-super-cluster . . . . . 9 5.1. Vai trò của Controller System . . . . . 9 5.2. Partition ở mức top-level . . . . . 10 Bảng 5-2. Quy tắc partition top-level khuyến nghị . . . . . 10 6. Super-cluster và subcluster 10 Bảng 6-1. Vai trò của 4 subcluster trong mô hình xoay vòng 4 pha . . . . . 11 6.1. Interface tối thiểu của một subcluster . . . . . 11 Bảng 6-2. Interface logic tối thiểu của subcluster . . . . . 11 7. GLB_BANK: vi kiến trúc bộ nhớ cục bộ 12 Bảng 7-1. Cấu trúc GLB_BANK trong mỗi subcluster . . . . . 12 7.1. Bank input: modulo-3 row banking + 16-lane width . . . . . 12 7.2. Resident rows, Q_in và cơ chế row-slot vòng . . . . . 13 7.3. Bank weight: 3 reduction lane thay vì 16 subbank . . . . . 13 Bảng 7-2. Ý nghĩa của 3 bank_weight theo từng mode . . . . . 13 7.4. Bank output: 4 stream output song song . . . . . 14 7.5. Nguyên tắc dung lượng cho compiler . . . . . 14 8. ROUTER_CLUSTER: điều hướng dữ liệu cục bộ và liên-cluster 14 8.1. Tách ba đường logic: input, weight và psum . . . . . 15 Bảng 8-1.

Ba họ router trong mỗi subcluster . . . . . 15 8.2. Router profile cần có những trường gì . . . . . 15 Bảng 8-2. Các trường logic cần có trong ROUTER_PROFILE . . . . . 15 8.3. Các pattern điều hướng điển hình . . . . . 16Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 3 9. PE_CLUSTER: khối tính toán trung tâm 16 9.1. Cửa sổ dữ liệu ngang là phần không được bỏ quên . . . . . 16 Bảng 9-1. Các mode cửa sổ ngang cần có . . . . . 16 9.2. Các mode tính toán của PE . . . . . 17 Bảng 9-2. Chế độ hoạt động đề xuất của PE_CLUSTER . . . . . 17 9.3. Dataflow minh họa cho conv3×3 stride 1 và stride 2 . . . . . 17 10. PPU: post-processing unit cho psum, activation và tensor ops nhẹ 18 Bảng 10-1. Hai lớp chức năng của PPU . . . . . 18 11. Descriptor-driven execution model 18 11.1. CPU/compiler phải làm gì . . . . . 19 11.2. IP phải làm gì . . . . . 19 11.3. Vì sao descriptor phải ở mức layer và tile . . . . . 19 Bảng 11-1. Phân vai giữa các descriptor . . . . . 20 12. Định dạng nhị phân của descriptor 20 12.1. NET_DESC (64 byte) . . . . . 20 Bảng 12-1. Đề xuất format NET_DESC . . . . . 20 12.2. LAYER_DESC (64 byte) . . . . . 20 Bảng 12-2. Đề xuất format LAYER_DESC . . . . . 21 12.3. TILE_DESC (64 byte) . . . . . 22 Bảng 12-3. Đề xuất format TILE_DESC . . . . . 22 12.4. Cờ khuyến nghị cho TILE_DESC . . . . . 22 Bảng 12-4. Bitfield cờ tile khuyến nghị . . . . . 22 12.5. ROUTER_PROFILE và POST_PROFILE . . . . . 23 Bảng 12-5. Đề xuất ROUTER_PROFILE 16 byte . . . . . 23 Bảng 12-6. Đề xuất POST_PROFILE 16 byte . . . . . 23 13. Đặc tả thanh ghi (register map) cho Controller System và các cluster 23 13.1. Global CSR ở mức chip . . . . . 23 Bảng 13-1. Global CSR . . . . . 24 13.2. CSR của descriptor fetch và barrier manager . . . . . 24 Bảng 13-2. Descriptor fetch và barrier manager . . . . . 24 13.3. CSR của mỗi super-cluster . . . . . 24 Bảng 13-3. CSR của mỗi super-cluster (base = 0x1000 + sc_id*0x1000) . . . . . 24 13.4. Shadow register nội bộ của subcluster . . . . . 25 Bảng 13-4. Nhóm shadow register nội bộ của subcluster . . . . . 25 13.5. Gợi ý memory map MMIO tổng thể . . . . . .

. . . . . 25 14. Layout dữ liệu, địa chỉ src/dst và cơ chế swizzle 25 14.1. Ba lớp địa chỉ . . . . . 26 14.2. Địa chỉ input . . . . . 26 14.3. Địa chỉ weight . . . . . 26Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 4 14.4. Địa chỉ output . . . . . 26 14.5. Swizzle output input . . . . . 26 14.6. Concat, upsample và residual trên cùng một hạ tầng . . . . . 27 Bảng 14-1. Cách thực hiện các op tensor-aware . . . . . 27 15. Flow compiler / pre-compute step by step 27 15.1. Chọn partition như thế nào . . . . . 28 Bảng 15-1. Quy tắc chọn partition được chốt trong handbook . . . . . 28 15.2. Thuật toán chọn tile . . . . . 28 15.3. Cách tính src/dst cho từng tile . . . . . 28 15.4. Pseudocode compile cho một layer . . . . . 29 16. Runtime controllers và FSM 29 16.1. FSM mức tile . . . . . 29 Bảng 16-1. Ý nghĩa từng trạng thái của FSM tile-level . . . . . 29 16.2. Scheduler 4 pha trong super-cluster . . . . . 30 Bảng 16-2. Luật ưu tiên arbiter của cổng ngoài trong một super-cluster . . . . . 30 16.3. Barrier và dependency cho nhánh concat/skip . . . . . 30 17. Ánh xạ layer-by-layer của YOLOv10n lên accelerator 31 Bảng 17-1. Mapping layer/stage của YOLOv10n . . . . . 31 18. Hai ví dụ worked-example để kiểm tra spec 31 18.1. Ví dụ A: Conv3×3 stride 2 ở backbone . . . . . 31 18.2. Ví dụ B: Head Upsample + Concat + fusion conv . . . . . 32 19. Decomposition RTL: module nào cần viết 33 Bảng 19-1. Bộ module RTL khuyến nghị cho V1 . . . . . 33 19.1. Gợi ý interface SystemVerilog ở mức module . . . . . 33 19.2. Pipeline nội bộ của subcluster . . . . . 34 19.3. Những nơi nên thêm assertion từ đầu . . . . . 35 20. Kế hoạch verify, chỉ số đo và lộ trình bring-up 35 Bảng 20-1. Lộ trình verify khuyến nghị . . . . . 35 20.1. Bộ testcase tối thiểu phải có . . . . . 36 20.2. Chỉ số nên đo ngay từ V1 . . . . . 36 Bảng 20-2. Performance / efficiency counters nên có . . . . . 36 20.3. Lộ trình silicon V1 V2 . . . . . 36 21. Kết luận thực thi 37 Phụ lục A. Ví dụ descriptor cho một layer Conv3×3 stride 2 37 Bảng A-1. Ví dụ các trường trọng yếu của LAYER_DESC . . . . . 37 Phụ lục B. Thuật ngữ và chữ viết tắt 38 Bảng B-1. Thuật ngữ dùng xuyên suốt handbook . . . . . 38 References 38

descriptor-driven Trang 5 1. Mục tiêu, phạm vi và triết lý tài liệu Handbook này trả lời trực tiếp bài toán thiết kế một accelerator cố định phần cứng cho YOLOv10n nhưng được điều khiển bằng descriptor, nghĩa là phần cứng chỉ chứa các primitive, các bank SRAM, router, PE và các FSM cần thiết; còn toàn bộ quyết định về chia tile, chọn mode, chọn partition, tính địa chỉ load/store, số pass và lịch xoay vòng buffer sẽ được tiền tính trên CPU/compiler rồi phát vào IP dưới dạng descriptor. Đây là bước đi tiếp theo của báo cáo kiến trúc trước đó [A1]. Phạm vi tài liệu không dừng ở mô tả ý tưởng mà đi tới một mức có thể dùng làm mốc để viết mã RTL. Vì vậy tài liệu sẽ đặc tả: kiến trúc top-level của chip; tổ chức 4 super-cluster theo lưới 2×2; vi kiến trúc của GLB_BANK, ROUTER_CLUSTER, PE_CLUSTER, PPU; định dạng và ý nghĩa của LAYER_DESC/TILE_DESC/ROUTER_PROFILE/POST_PROFILE; bộ thanh ghi điều khiển; FSM mức tile và mức layer; decomposition RTL; và kế hoạch verify. Nguyên tắc chủ đạo là tối thiểu hóa logic 'ra quyết định' trong IP. Mọi thứ có thể giải được bằng offline compile đều nên giải trên CPU. Do đó IP của tài liệu này không có những block tính ràng buộc nặng về thời gian chạy, không có block tìm tile tối ưu tại runtime, và cũng không cố đoán layout. IP chỉ thực hiện descriptor đã được xác định, dùng các counter và comparator nhỏ để bước qua tile, pass và trạng thái. • Mục tiêu 1: giữ được tinh thần RS / RS+ và HM-NoC của Eyeriss/Eyeriss v2 nhưng rút gọn về một dạng có thể hiện thực thực tế cho YOLOv10n [R6], [R7]. • Mục tiêu 2: bao phủ được các primitive thực sự xuất hiện trong yolov10n.yaml thay vì giả thiết sai rằng mô hình chỉ có conv3x3/1x1 [R2]. • Mục tiêu 3: hệ thống hóa một bộ descriptor đủ chặt để driver phần mềm, controller trên chip và testbench cùng hiểu một ngôn ngữ điều khiển thống nhất. • Mục tiêu 4: tạo một đường đi rõ ràng từ nghiên cứu spec RTL verify bring-up. 1.1. Bài toán được chốt như thế nào Đầu vào của toàn bộ hệ thống là một graph YOLOv10n đã biết trước. CPU/compiler sẽ hạ graph này xuống một tập primitive phần cứng cố định, tạo layout tensor, chia tile theo dung lượng phần cứng, gán các tile lên 4 super-cluster và 4 subcluster nội bộ, rồi phát descriptor. Controller system chỉ cần fetch descriptor, nạp shadow register, kích hoạt DMA/load/store, lập lịch router và cho PE chạy theo template tương ứng. Điểm rất quan trọng là cách làm này không biến accelerator thành một CPU hoặc một VLIW processor. Đây vẫn là một IP chuyên dụng: tập primitive là hữu hạn, luồng dữ liệu là hữu hạn, các register là hữu hạn. Descriptor ở đây không phải ISA từng chu kỳ mà là macro-command cho cả một layer hoặc một tile. Nếu giữ đúng triết lý này, độ phức tạp của phần cứng nằm ở datapath và ở quản lý bộ nhớ cục bộ, chứ không nằm ở một scheduler tổng quát kiểu phần mềm. Đây là điểm then chốt để IP vừa đủ linh hoạt cho YOLOv10n nhưng vẫn synthesize và verify được. 1.2. Những giả định nền • Batch inference = 1. • Đường dữ liệu cơ sở: input/weight INT8; activation sau requant INT8 hoặc INT16; psum giữ bề rộng lớn hơn, ví dụ 20 bit hoặc hơn tùy budget [A1]. • Mỗi super-cluster có 1 external port 128 bit/cycle; toàn chip có 4 super-cluster nên tổng băng thông đỉnh từ ngoài vào là 512 bit/cycle nếu hệ thống ngoài cấp đủ.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 6 • Mỗi super-cluster chứa 4 subcluster; mỗi subcluster

có 1 GLB_BANK, 1 ROUTER_CLUSTER, 1 PE_CLUSTER 3×4; bank_input có 3 bank, mỗi bank 16 subbank theo chiều width; bank_weight có 3 bank; bank_output có 4 bank [A1]. • Mục tiêu silicon V1 ưu tiên các primitive: RS_DENSE_3x3, OS_1x1, DW_3x3, MAXPOOL_5x5, MOVE/CONCAT/UPSAMPLE, EWISE_ADD, detect packaging và GEMM_ATTN cơ bản [A1]. 2. Cơ sở nghiên cứu và các tham chiếu phải bám Toàn bộ handbook này được xây trên hai lớp nguồn. Lớp thứ nhất là báo cáo kiến trúc đã tổng hợp trước đó [A1], trong đó đã chuẩn hóa super-cluster/subcluster, modulo-3 input banking, 4-bank output, swizzle engine, ring-buffer 4 pha và các primitive tối thiểu. Lớp thứ hai là các nguồn gốc về mô hình YOLOv10 và về dataflow/NoC của Eyeriss. Các nguồn mô hình dùng để xác minh chính xác YOLOv10n hiện có những khối nào, tránh thiết kế thừa hoặc thiếu. Các nguồn accelerator dùng để kiểm chứng các quyết định về row-stationary, NoC phân cấp và định hướng reuse. Bảng 2-1. Bộ tham chiếu chính dùng xuyên suốt tài liệu Mã Nguồn Vai trò trong handbook [A1] Báo cáo kiến trúc YOLOv10n đã tải lên Điểm xuất phát nội bộ; chốt các giả định về GLB_BANK/ROUTER/PE, modulo-3, swizzle, scheduler và primitive. [R1] THU-MIG/yolov10 (repository) Nguồn chính thức để kiểm tra cấu trúc mô hình và triển khai module. [R2] yolov10n.yaml Xác định chính xác backbone/head của YOLOv10n. [R3] ultralytics/nn/modules /block.py Xác định chi tiết SCDown, SPPF, PSA, CIB, RepVGGDW, C2fCIB. [R4] YOLOv10 paper (arXiv 2405.14458) Xác định triết lý consistent dual assignments, SCDown, CIB, PSA và định hướng mô hình. [R5] Ultralytics YOLOv10 docs Xác nhận đầu suy luận one-to-one và NMS-free inference. [R6] Eyeriss row-stationary paper Nền tảng RS dataflow và ưu tiên giảm data movement. [R7] Eyeriss v2 / HM-NoC paper Nền tảng NoC phân cấp, RS+ và khả năng thích nghi layer đa dạng. Theo yolov10n.yaml chính thức, backbone/head của YOLOv10n gồm các module Conv, C2f, SCDown, SPPF, PSA, C2fCIB, nn.Upsample, Concat và v10Detect [R2]. block.py cho thấy SCDown thực chất là pointwise 1×1 rồi depthwise stride conv; SPPF là 1×1 và MaxPool5×5 lặp; PSA tách kênh, chạy attention trên một nửa kênh rồi ghép lại; CIB dùng depthwise + pointwise; còn RepVGGDW có hai nhánh DW7×7 và DW3×3 có thể fuse ở inference [R3]. Do đó accelerator không được hard-wire chỉ cho conv3×3 chuẩn. YOLOv10 paper và tài liệu Ultralytics đều nhấn mạnh rằng huấn luyện dùng consistent dual assignments, còn suy luận dùng one-to-one head để bỏ NMS [R4], [R5]. Điều này có nghĩa phần cứng không cần một NMS engine như các pipeline detect cổ điển. Phần post-processing cuối chỉ cần packaging/formatting phù hợp với tensor của head. 3. YOLOv10n nhìn từ góc phần cứng 3.1. Primitive thực sự cần support Từ cấu trúc YAML và mã nguồn module, bài toán phần cứng của YOLOv10n có thể hạ xuống một tập primitive hữu hạn. Tập này không cần quá lớn, nhưng phải đủ bao phủ các khối xuất hiện thật sự trong mô hình. Quan trọng nhất là tách bạch các primitive 'tính toán nặng' khỏi các primitive 'di chuyển / biến đổi tensor'.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 7 Bảng 3-1. Ánh xạ primitive phần cứng cho các module chính của YOLOv10n Module YOLOv1 0n Primitive

phần cứng Ghi chú Conv 3×3 RS_DENSE_3x3 Primitive chủ lực cho backbone/head spatial conv. Conv 1×1 OS_1x1 Projection, split/merge, QKV, FFN, detect head. SCDown OS_1x1 + DW_3x3 Pointwise trước, rồi depthwise stride-2 [R3], [R4]. SPPF OS_1x1 + MAXPOOL_5x5 + CONCAT + OS_1x1 Không phải Conv5×5; cần comparator path thay vì MAC path [R3]. PSA OS_1x1 + GEMM_ATTN + DW_3x3 + OS_1x1 Attention chỉ chạy trên một nửa kênh [R3]. C2f OS_1x1 + RS_DENSE_3x3 + CONCAT + OS_1x1 Bottleneck nội bộ chủ yếu là 3×3. C2fCIB OS_1x1 + CIB(DW/1×1[/DW7×7]) + CONCAT + OS_1x1 V1 có thể để DW7×7 là multi-pass. Upsampl e UPSAMPLE / MOVE Không cần MAC; nhân đôi địa chỉ hoặc duplicate write. Concat CONCAT / MOVE Đổi channel offset, giữ nguyên chỉ số (h,x). Residual add EWISE_ADD Dùng PPU hoặc adder trong router/output path. v10Detec t RS_DENSE_3x3 + OS_1x1 + detect packaging Không cần NMS engine ở inference [R5]. Từ góc phần cứng, quyết định rất quan trọng là SPPF phải được hiểu là MaxPool5×5 lặp, không phải Conv5×5. Điều này giúp tránh thiết kế sai một datapath conv5×5 phức tạp mà mô hình V1 thực ra chưa cần. Tương tự, PSA tuy là attention nhưng chỉ nằm trên stage sâu với feature map nhỏ, nên có thể chấp nhận một datapath GEMM/vector unit đơn giản hơn và không cần tối ưu bằng conv3×3. 3.2. Các đặc điểm khiến YOLOv10n không thể chạy tối ưu bằng một dataflow duy nhất • Spatial conv 3×3 cần line-buffer theo chiều height và window reuse theo kernel rows. • Pointwise 1×1 có tính chất compute khác hẳn: reuse chủ yếu theo kênh, không theo kernel rows. • Depthwise và grouped conv có reuse kênh thấp; chia việc theo group/channel thường hiệu quả hơn chia theo Cin rồi reduce. • SPPF, concat, upsample, residual là các op điều khiển dữ liệu; nếu cố ép vào MAC array sẽ lãng phí công suất. • PSA có GEMM, softmax, và positional depthwise; cần tách thành chuỗi primitive thay vì một datapath attention lớn. • Detect head có nhiều nhánh và yêu cầu packaging tensor cuối; bản thân nó không chỉ là conv thuần. Kết luận trực tiếp là accelerator phải là một compute fabric cố định nhưng có thể đổi mode. Ta không nên gắn cứng cả chip vào duy nhất một cách đi dữ liệu. Hướng đúng là giữ một số datapath con cố định, rồi chọn mode bằng descriptor. Nói ngắn gọn: YOLOv10n vẫn là mô hình thiên về convolution, nhưng không phải mô hình chỉ gồm convolution. Cái cần là một accelerator 'convolution-centric but tensor-aware'. 3.3. Những gì không nên làm ở silicon V1Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 8 1. Không hard-wire Conv5×5 như primitive bắt buộc. Nếu sau này cần, có thể làm multi-pass hoặc ghép cluster; nhưng V1 nên ưu tiên SPPF và Conv/DW thực sự xuất hiện trong YOLOv10n. 2. Không thiết kế NMS engine. YOLOv10 inference chuẩn sử dụng one-to-one head để bỏ NMS [R4], [R5]. 3. Không cố làm một scheduler runtime tổng quát. CPU/compiler phải gánh phần tìm tile, tìm partition và phát descriptor. 4. Không ép mọi layer phải chia theo Cin. Cần hỗ trợ ít nhất GROUP_SPLIT, COUT_SPLIT và CIN_SPLIT; ưu tiên khác nhau tùy primitive [A1]. 5. Không coi 4 subcluster chỉ là ping-pong kép. Với head của YOLOv10n, luôn cần nơi giữ skip tensor, tensor sau upsample hoặc tensor đang drain. 4. Mục tiêu phần cứng và triết lý thiết kế Triết lý phần cứng của handbook

này kế thừa báo cáo gốc [A1] và được đẩy tới mức cụ thể hơn: tối ưu reuse, giữ compute liên tục và tái cấu hình theo primitive. Ba mục tiêu đó tương ứng với ba khối logic lớn của chip: GLB_BANK để giữ reuse cục bộ, ROUTER_CLUSTER để đảm bảo đường đi dữ liệu linh hoạt nhưng hữu hạn, và PE_CLUSTER để thực hiện các mode tính toán chính. Bảng 4-1. Ba trục tối ưu của accelerator Trục tối ưu Ý nghĩa phần cứng Hệ quả thiết kế Reuse dữ liệu Giảm số lần đọc/ghi tầng nhớ đắt tiền Cần modulo-3 line-buffer, weight staging, psum cục bộ và swizzle on-chip. Dòng chảy liên tục Không đợi DRAM từng chu kỳ Cần ring-buffer 4 pha, prefill resident rows, overlap compute/fill/drain. Tái cấu hình theo layer Không hard-wire cho conv3×3 duy nhất Cần mode bit cho PE, router profile, post-profile và descriptor-driven scheduling. Nếu dùng ngôn ngữ kiến trúc hệ thống, chip này là một accelerator descriptor-driven, spatial, cluster-based, memory-centric. Spatial vì compute được triển khai trên mảng PE; cluster-based vì toàn chip được chia lưới 2×2 super-cluster, mỗi super-cluster lại có 4 subcluster; memory-centric vì hiệu năng thật sự bị quyết định bởi tổ chức banking, swizzle và overlap của dữ liệu hơn là bởi số MAC thô. Đây là điểm khác biệt giữa một accelerator 'trông mạnh trên giấy' và một accelerator chạy được mô hình thật. Nếu memory map hoặc swizzle sai, throughput MAC sẽ không cứu được hệ thống. 4.1. Nguyên tắc chuẩn để chọn mode 1. Chọn primitive trước, rồi mới chọn partition, tile và router. 2. Ưu tiên giữ full width theo bội 16; chỉ giảm channel tile khi capacity không đủ; chỉ giảm width khi channel tile đã quá nhỏ [A1]. 3. Ưu tiên GROUP_SPLIT cho depthwise/grouped, COUT_SPLIT cho pointwise/projection, và CIN_SPLIT chỉ khi memory ép buộc [A1]. 4. Đầu ra của mọi primitive phải có đường đi rõ ràng: về bank_output để giữ psum/act, sang swizzle để trở thành bank_input của layer sau, hoặc spill ra external arena. 5. Mỗi descriptor phải đủ để phần cứng thi hành mà không cần suy luận thêm. 5. Kiến trúc top-level của chipHandbook thiết kế accelerator YOLOv10n descriptor-driven Trang 9 Top-level của chip được chốt là một lưới 2×2 super-cluster. Mỗi super-cluster có một external port 128 bit/cycle riêng, vì vậy tổng cộng toàn chip có bốn cổng 128 bit/cycle. Cấu hình này khớp với yêu cầu mới của bạn: 4 cluster lớn, mỗi cluster lớn có 4 subcluster, và các cluster lớn được nối theo hướng trên/dưới/trái/phải để chia việc, multicast dữ liệu và reduce psum. +----------+ | Controller System | | fetch / decode / barrier / DMA | +----------+----------+ | +----------+----------+ | | | Port0 128b Port1 128b Perf / IRQ | | +----------+ <==== fabrics ====> +----------+ | SC(0,0) | | SC(0,1) | | 4 subcluster | | 4 subcluster | +----------+ +----------+ ^ ^ | | +----------+ <==== fabrics ====> +----------+ | SC(1,0) | | SC(1,1) | | 4 subcluster | | 4 subcluster | +----------+ +----------+ | | Port2 128b Port3 128b Hình 5-1. Tổ chức top-level đề xuất cho chip với 4 super-cluster theo lưới 2×2. Có ba fabric logic ở mức top-level. Thứ nhất là fabric IACT để multicast input activation, halo row, skip tensor và tensor sau upsample. Thứ hai là fabric WGT để multicast weight theo hướng phù hợp, đặc biệt hiệu quả cho pointwise hoặc khi hai super-cluster cùng tiêu thụ một tile weight. Thứ ba là fabric PSUM để reduce partial sum

khi bắt buộc phải chia theo Cin hoặc khi muốn gộp kết quả theo chiều dọc. Bảng 5-1. Khuyến nghị về các fabric liên-super-cluster Fabric Chiều ưu tiên Độ rộng khuyến nghị Vai trò IACT fabric N/S/E/W 128 bit/cycle Chuyển input/halo/skip/upsample giữa các super-cluster. WGT fabric E/W là chính 128 bit/cycle Multicast weight/projection tile tới cluster lân cận. PSUM fabric N/S là chính vector_width × psum_bits Reduce partial sum giữa các phần split theo Cin hoặc reduce dọc. Với vector width = 16 và psum width giả sử 20 bit, một vector psum đầy đủ rộng 320 bit. Vì vậy nếu fabric PSUM muốn chuyển nguyên vector theo một beat, nó phải rộng 320 bit. Còn nếu area bị siết chặt, có thể time-multiplex theo 2 hoặc 4 beat với chi phí latency. Trong handbook này tôi giữ fabric PSUM như một tham số thiết kế: chức năng là bắt buộc, còn physical width có thể trade-off theo budget. 5.1. Vai trò của Controller System • Fetch descriptor từ instruction/descriptor RAM hoặc từ external memory qua DMA. • Decode LAYER_DESC/TILE_DESC và nạp shadow register vào super-cluster tương ứng. • Điều phối barrier và dependency giữa các nhánh concat/skip/upsample. • Quản lý con trỏ descriptor, pointer của tensor arena và IRQ/performance counters.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 10 • Phân quyền truy cập external port cho từng subcluster nội bộ thông qua scheduler của mỗi super-cluster. • Thu thập trạng thái done/error và đưa ra cơ chế recover hoặc halt. Controller System không phải datapath tính toán. Nó là một control-plane: hiểu descriptor, hiểu dependency, cấp lệnh cho DMA/scheduler và theo dõi trạng thái. Nếu mô tả theo RTL, đây là khối gồm descriptor fetch engine, descriptor decoder, scoreboard/barrier manager, global scheduler, performance monitor và CSR/MMIO block. 5.2. Partition ở mức top-level Ở mức 4 super-cluster, có ba cách cắt việc lớn nhất: cắt theo chiều height của feature map, cắt theo output-channel, hoặc cắt theo input-channel/group. Cách nào tối ưu phụ thuộc primitive. Một quy tắc thực dụng là: spatial conv ở các stage phân giải cao ưu tiên chia theo height giữa hai hàng super-cluster; pointwise/projection ưu tiên chia theo output-channel giữa hai cột super-cluster; depthwise/grouped ưu tiên group-split để giảm reduce; còn Cin-split chỉ dùng khi không đủ bộ nhớ để giữ tile input lớn. Bảng 5-2. Quy tắc partition top-level khuyến nghị Primitive Partition top-level ưu tiên Lý do RS_DENSE_ 3x3 H_SPLIT + COUT_SPLIT Giảm resident rows mỗi cluster và tránh reduce liên-cluster khi có thể. OS_1x1 COUT_SPLIT Input có thể multicast; mỗi cluster xuất một nhóm output-channel độc lập. DW_3x3 / DW_7x7 GROUP_SPLIT Mỗi cluster xử lý nhóm channel độc lập, không cần reduce psum. MAXPOOL_ 5x5 H_SPLIT Chủ yếu là spatial window, ít lợi ích khi cắt theo kênh. PSA/GEMM COUT_SPLIT hoặc H_SPLIT trên feature map nhỏ Giữ tensor attention nhỏ và giảm traffic. MOVE/CON CAT/UPSA MPLE Theo tensor locality Giữ tensor gần nơi tiêu thụ để giảm copy xa. 6. Super-cluster và subcluster Một super-cluster là đơn vị cân bằng giữa băng thông bên ngoài, tính cục bộ của bộ nhớ và độ phức tạp của điều phối. Mỗi super-cluster gồm 4 subcluster. Bên trong một super-cluster, 4 subcluster nên được đặt theo lưới 2×2 để

ROUTER_CLUSTER có thể hỗ trợ các pattern local unicast, grouped multicast, interleaved multicast và broadcast ngắn giống tinh thần HM-NoC [R7].Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 11 External Port 128b/cycle | +----------+ | Arbiter | +----------+ _____|___|_____ / | \ / | \ +----------+ +----------+ +----------+ +----------+ | Subcluster 0 | | Subcluster 1 | | Subcluster 2 | | Subcluster 3 | | GLB -> RTR -> | | GLB -> RTR -> | | GLB -> RTR -> | | GLB -> RTR -> | | PE3x4 | | PE3x4 | | PE3x4 | | PE3x4 | +----------+ +----------+ +----------+ +----------+ ^ ^ ^ ^ ^ ^ ^ ^ | |_____| |_____| |_____| | | local fabrics (input / weight / psum / swizzle) | +----------+ Hình 6-1. Tổ chức logic của một super-cluster. Điểm quan trọng ở đây là external port 128 bit/cycle được dùng chung bởi 4 subcluster trong cùng một super-cluster. Vì vậy local arbiter/scheduler bắt buộc phải có. Nó không chỉ chọn ai được load ở chu kỳ hiện tại mà còn phải hiểu vai trò RUNNING/FILLING/DRAINING/HOLD_SKIP để overlap dữ liệu với tính toán. Bảng 6-1. Vai trò của 4 subcluster trong mô hình xoay vòng 4 pha Vai trò Mô tả Loại traffic chính RUNN ING Đang thực thi tile hiện tại Đọc bank_input/weight, ghi bank_output/psum. FILLIN G Đang prefetch tile kế tiếp Load input/weight từ port ngoài hoặc fabric. DRAIN ING Đang swizzle/store output của tile trước Đọc bank_output, ghi bank_input đích hoặc spill ra ngoài. HOLD _SKIP Giữ skip tensor, upsample tensor hoặc weight cho layer sau Đọc/ghi nhẹ, ưu tiên tính sẵn sàng hơn throughput. Cơ chế này vượt ra ngoài ping-pong buffer truyền thống. Trong head của YOLOv10n, đặc biệt ở các đoạn Upsample + Concat + fusion conv, việc luôn có một subcluster rảnh để giữ skip tensor làm giảm đáng kể traffic ra ngoài. 6.1. Interface tối thiểu của một subcluster Bảng 6-2. Interface logic tối thiểu của subcluster Tên interface Hướng Nội dung cfg_* in Shadow register do controller nạp: mode, tile, base addr, router profile, post profile. ext_load_* in Beat dữ liệu 128 bit/cycle từ external port qua arbiter. ext_store_ * out Beat dữ liệu ra ngoài khi spill hoặc writeback. nbr_in_* in Dữ liệu từ subcluster lân cận hoặc từ super-cluster lân cận qua fabric. nbr_out_* out Dữ liệu gửi tới subcluster/super-cluster khác. done/error /status out Trạng thái local để scheduler và controller system theo dõi. perf_* out Counter utilization, beat count, stall count, overflow/error count.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 12 7. GLB_BANK: vi kiến trúc bộ nhớ cục bộ GLB_BANK là trái tim của mỗi subcluster. Trong handbook này, GLB_BANK không được xem như một SRAM thô mà là một local memory subsystem có cấu trúc rõ ràng: bank_input[3], bank_weight[3], bank_output[4], weight staging register/FIFO, output readback FIFO, metadata RAM cho slot/valid và một swizzle engine. Mọi quyết định về hiệu năng bền vững đều hội tụ ở đây. Bảng 7-1. Cấu trúc GLB_BANK trong mỗi subcluster Khối Số lư ợng Vai trò Nhận xét hiện thực bank_i nput 3 Lưu activation input; line-buffer theo chiều height Mỗi bank có 16 subbank, khớp 128 bit/cycle cho INT8. bank_ weigh t 3 Lưu weight cho 3 reduction lane Không cần 16 subbank như input; thêm staging register để giảm áp SRAM. bank_ outpu t 4 Lưu psum hoặc

activation cuối Khớp tự nhiên với 4 cột PE; hỗ trợ PSUM mode và ACT mode. metad ata RAM 1 cụm Giữ valid, slot mapping, base_h/base_c, trạng thái ring Cho phép circular row-slot và kiểm tra resident rows. swizzl e engi ne 1 Đổi layout output input cho layer sau Bắt buộc nếu muốn xâu chuỗi nhiều layer on-chip. local a rbiter 1 Điều phối external port và các đường đọc/ghi nội bộ Làm việc cùng scheduler của super-cluster. 7.1. Bank input: modulo-3 row banking + 16-lane width Cấu trúc bank_input là điểm đặc trưng nhất của toàn bộ thiết kế. Ý tưởng gốc của bạn là bank_input_0 giữ các dòng 0,3,6..., bank_input_1 giữ các dòng 1,4,7..., bank_input_2 giữ các dòng 2,5,8... để nuôi conv3×3 liên tục. Handbook chuẩn hóa ý tưởng đó thành một quy luật tổng quát: bank_input = h mod 3 và row_slot = floor(h / 3). Như vậy 'bank_input_0 chứa dòng 0 và 3' không còn là một trường hợp đặc biệt mà trở thành hệ quả tự nhiên của modulo-3 row banking [A1]. Mỗi bank_input lại có 16 subbank theo chiều width. Điều này có nghĩa một beat 128 bit có thể mang 16 phần tử 8 bit song song, ứng với 16 vị trí liên tiếp theo width của cùng một dòng và cùng một channel. Nói theo pointer logic gốc, nếu ptn0 là block width, thì x = ptn0 × 16 + lane, với lane [0..15]. logical coordinate: h = ptn2 c = ptn1 x = ptn0 * 16 + lane physical mapping: ib = h mod 3 row_slot = floor(h / 3) mod Q_in subbank = lane xb = floor(x / 16) address: addr_in = ((row_slot * Ctile) + c) * Wblk + xb Hình 7-1. Công thức ánh xạ input từ tọa độ logic sang bank_input. Công thức trên là dạng 'đủ cứng để code RTL' nhưng vẫn đủ đơn giản để compiler tiền tính. Ở runtime, hardware không cần nhân chia tổng quát; compiler có thể phát trực tiếp các giá trị base_h, row_slot_base, Wblk, Ctile và các bound cần so sánh. Phần cứng chỉ còn phép cộng và so sánh trên counter.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 13 Ý nghĩa thực tế của modulo-3 row banking là biến GLB input thành một line-buffer tổng quát cho mọi kernel cao Kh, không chỉ riêng conv3×3. Đây là lý do thiết kế này vẫn phục vụ được DW7×7 multi-pass hoặc maxpool5×5 mà không cần đổi tổ chức bank. 7.2. Resident rows, Q_in và cơ chế row-slot vòng Nếu PE cluster có 4 cột PE cùng xử lý 4 output-row song song, số lượng input row phải resident đồng thời được tính bởi công thức: R_need = Kh_eff + (Pcol - 1) * stride_h = Kh_eff + 3 * stride_h Q_in = ceil(R_need / 3) R_new = Pcol * stride_h = 4 * stride_h Hình 7-2. Công thức resident rows và lượng dòng mới phải nạp sau mỗi micro-tile. • Với conv3×3 stride 1: R_need = 6, Q_in = 2, tức mỗi bank_input cần 2 row-slot. • Với conv3×3 stride 2: R_need = 9, Q_in = 3, tức mỗi bank_input cần 3 row-slot. • Với conv1×1 stride 1: R_need = 4, Q_in = 2; vẫn dùng modulo-3 nhưng 3 PE row sẽ đổi vai trò thành reduction slice theo kênh. • Với maxpool5×5 stride 1: R_need = 8, Q_in = 3. • Với depthwise7×7 stride 1: R_need = 10, Q_in = 4. Khi tile chạy xong một đợt 4 output-row, lượng dòng mới cần nạp thêm là R_new = 4 × stride_h. Thay vì dịch chuyển dữ liệu bên trong SRAM, GLB_BANK phải dùng cơ chế circular row-slot: row_slot được wrap theo modulo Q_in. Đây là cách duy nhất sạch về RTL, vì nó loại bỏ memcopy nội bộ và biến line-buffer thành một ring-buffer được điều khiển bằng metadata. for each logical row h to be loaded: ib = h mod 3 row_slot = floor(h / 3) mod Q_in write bank_input[ib][row_slot][...] Hình 7-3. Ý

tưởng vòng row-slot giúp không phải dời dữ liệu trong SRAM. 7.3. Bank weight: 3 reduction lane thay vì 16 subbank bank_weight khác bank_input ở chỗ nó không cần vector hóa theo width. Một weight của conv có thể được broadcast để dùng cho cả 16 lane theo chiều width. Vì vậy cấu trúc đúng là 3 bank weight tương ứng với 3 reduction lane của PE row, còn bên trong mỗi bank dùng address linear hoặc banking nhỏ theo nhu cầu timing, chứ không cần 16 subbank đối xứng với input. Bảng 7-2. Ý nghĩa của 3 bank_weight theo từng mode Mode Ý nghĩa bank_weight[0..2] RS_DENSE_3x3 Ba hàng kernel row 0/1/2. OS_1x1 Ba lát reduction slice theo input-channel để tận dụng 3 PE row. DW_3x3 Ba hàng kernel row cho một depthwise channel/group. DW_7x7 multi-pass Ba hàng kernel row cho pass 0..2; pass tiếp theo nạp hàng tiếp. MAXPOOL_5x5 Không dùng như weight; datapath comparator có thể bypass bank_weight. GEMM_ATTN Có thể tái sử dụng như local matrix tile store nếu muốn giữ thiết kế gọn.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 14 RepVGGDW trong block.py có thể fuse nhánh DW7×7 và DW3×3 thành một kernel duy nhất ở inference [R3]. Điều này rất thuận lợi cho V1: compiler có thể fuse offline, sau đó hardware chỉ cần chạy một primitive DW_7x7 multi-pass. Tức là phần cứng không cần đồng thời giữ hai nhánh riêng. Giữa bank_weight và router_weight nên có weight staging register/FIFO. Vai trò của nó là giảm áp lực đọc SRAM mỗi chu kỳ, hỗ trợ multicast weight sang 4 cột PE và cho phép chồng lấp nạp weight cho pass kế tiếp. 7.4. Bank output: 4 stream output song song Nếu 4 cột PE tương ứng với 4 output-row song song thì 4 bank_output nên map tự nhiên theo 4 stream này. Đây là điểm khớp rất đẹp giữa compute organization và memory organization. Thay vì một bank output chung gây xung đột cổng, ta có 4 bank output tách biệt, mỗi bank gắn với một PE column. for pe_col in 0..3: h_out = h_base + pe_col * stride_h + tile_iter * (4 * stride_h) obank = pe_col oslot = floor((h_out - h_base) / (4 * stride_h)) mod Q_out addr = ((oslot * Cout_tile) + oc) * Wblk_out + xb bank_output[obank][addr] = data_or_psum Hình 7-4. Công thức ánh xạ output vào 4 bank_output. • PSUM mode: giữ partial sum trung gian khi còn pass theo Cin hoặc theo kernel. • ACT mode: giữ activation cuối sau bias, requant và activation. • Hỗ trợ thêm cho concat, upsample, residual add, PSA/GEMM tạm thời và detect head [A1]. Cần phân biệt rất rõ: bank_output không chỉ là nơi ghi kết quả conv cuối. Nó là vùng đệm trung tâm cho mọi op tensor-aware. Vì vậy thiết kế bank_output phải tính đến ít nhất hai kiểu độ rộng: psum rộng và activation hẹp. Có thể hiện thực bằng dual-bank riêng, hoặc một bank vật lý duy nhất nhưng với metadata mode và datapath pack/unpack phù hợp. Nếu không có bank_output đủ rõ vai trò PSUM/ACT, các op concat/upsample/residual sẽ buộc phải đi ra DRAM sớm. Điều đó triệt tiêu lợi ích của toàn bộ cụm subcluster. 7.5. Nguyên tắc dung lượng cho compiler Compiler phải chọn tile dưới các ràng buộc capacity rõ ràng. Với một subcluster: Input capacity constraint: Cap_in >= R_need * Wpad * Cin_tile * bytes_act Weight capacity constraint: Cap_w >= Wbuf(mode, Kh, Kw, Cin_tile, Cout_tile, pass) Output capacity constraint: Cap_out >= Q_out * Wblk_out * Cout_tile * bytes_out(mode)

Recommendation: 1) maximize Wtile first (align to 16) 2) then reduce Cin_tile / Cout_tile 3) only shrink Wtile if channel tiles become too small Hình 7-5. Bộ ràng buộc capacity mà compiler phải giải trước khi phát descriptor. 8. ROUTER_CLUSTER: điều hướng dữ liệu cục bộ và liên-clusterHandbook thiết kế accelerator YOLOv10n descriptor-driven Trang 15 ROUTER_CLUSTER là khối tạo nên sự linh hoạt thật sự của accelerator. Nếu GLB_BANK đảm nhiệm lưu giữ và PE_CLUSTER đảm nhiệm tính toán, thì ROUTER_CLUSTER quyết định dữ liệu sẽ đi đường nào, được broadcast tới đâu, có reduce ở đâu và có trở thành tensor của layer kế tiếp mà không quay ra DRAM hay không. Đây là chỗ tinh thần HM-NoC của Eyeriss v2 đi vào hiện thực [R7]. +----------+ bank_input[0..2] | | -> PE row/col lanes bank_weight[0..2]| ROUTER_CLUSTER | bank_output[0..3]| | -> local bank_output neighbor N/S/E/W| | -> neighbor N/S/E/W swizzle in | | -> swizzle / PPU +----------+ Hình 8-1. ROUTER_CLUSTER nhìn như một crossbar có profile cấu hình, không phải mạng tùy ý vô hạn. 8.1. Tách ba đường logic: input, weight và psum Kiến trúc đề xuất giữ ba họ router riêng: RIN cho activation/input, RWT cho weight, và RPS cho partial sum/output. Việc tách này giúp controller cấu hình đơn giản hơn, timing sạch hơn và phản ánh đúng tính chất reuse của từng loại dữ liệu. Bảng 8-1. Ba họ router trong mỗi subcluster Route r Số kênh Nguồn điển hình Đích điển hình RIN 3 bank_input, north/south/east/west, swizzle engine 12 PE, neighbor, concat/add path RWT 3 bank_weight, east/west weight fabric 3 PE row, neighbor right/left RPS 4 4 cột PE, local bank_output, neighbor vertical bank_output, PPU, south/north reduce fabric RIN phản ánh logic 'dữ liệu đầu vào có thể đến từ nhiều nơi': từ bank_input local, từ output đã swizzle của layer trước, từ skip tensor của nhánh khác hoặc từ super-cluster lân cận. RWT nhấn mạnh rằng weight thường broadcast theo chiều ngang hoặc theo row. RPS gắn với reduce theo cột và khả năng forward psum theo trục dọc. 8.2. Router profile cần có những trường gì Một sai lầm thường gặp là tả router bằng lời quá chung chung. Để viết RTL, cần chốt thành một profile bitfield. Tôi đề xuất một ROUTER_PROFILE tách được ba nhóm cấu hình: Bảng 8-2. Các trường logic cần có trong ROUTER_PROFILE Nhóm trường Mô tả RINx_SRC_S EL Chọn nguồn của từng router input: local bank, north, south, east, west, swizzle, bypass. RINx_DST_M ASK Bitmask chọn PE đích hoặc local consumer. RIN_BCAST_ MODE Kiểu broadcast: local, row, column, group, global. RWTx_SRC_S EL Nguồn weight cho từng row lane. RWT_HMCAS T_EN Bật/tắt multicast ngang trong cluster hoặc sang cluster bên cạnh. RPSx_DST_S EL Chọn đích của psum/output từ từng cột PE: bank_output, PPU, north/south, drop. RPS_ACC_M ODE Chế độ reduce: none, local accumulate, vertical accumulate, writeback-only.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 16 Nhóm trường Mô tả SWIZZLE_RO UTE Cho phép output đi qua swizzle engine rồi quay vào bank_input đích. Điểm hay của ROUTER_PROFILE là nó tách được cấu hình định kỳ của một primitive ra khỏi phần descriptor từng tile. Ví dụ conv3×3 stride 1 của cả một layer có thể dùng chung một profile router; chỉ tile descriptor mới thay đổi base addr, halo,

valid_w, valid_h và cờ biên. ROUTER_PROFILE nên sống như một bảng nhỏ riêng trong descriptor RAM. LAYER_DESC chỉ cần mang profile_id. Cách này giảm kích thước descriptor layer và giúp reuse cấu hình giữa nhiều tile. 8.3. Các pattern điều hướng điển hình 1. Local feed: bank_input local RIN PE. Đây là pattern phổ biến nhất cho conv sau khi tile đã resident. 2. Weight horizontal multicast: bank_weight local hoặc cluster trái RWT toàn bộ 4 cột của cùng một PE row. Pattern này rất phù hợp row-stationary. 3. Vertical psum reduce: cột PE RPS neighbor dưới hoặc trên. Dùng khi split theo Cin hoặc khi muốn xâu chuỗi reduction theo trục Y. 4. Swizzle-and-feed: bank_output swizzle bank_input của subcluster đích. Dùng cho layer kế tiếp, concat, upsample, reorder giữa backbone/head. 5. Tensor move: input hoặc output đi xuyên router mà không vào PE; dùng cho concat, move, bypass và các đường skip. Router phải được thiết kế sao cho không bắt compute chờ quá nhiều. Vì vậy mỗi router path cần ít nhất FIFO nhỏ hoặc skid buffer và handshake ready/valid. Perf counter nên đo stall theo từng họ router để compiler hoặc driver có thể thấy điểm nghẽn thật sự ở hệ thống. 9. PE_CLUSTER: khối tính toán trung tâm Từ các ví dụ conv3×3 stride 1/2 trong báo cáo trước, cách diễn giải chính xác nhất của PE cluster là: 3 hàng PE tương ứng 3 reduction lane; 4 cột PE tương ứng 4 output-row đang xử lý song song; còn 16 subbank width của bank_input thực chất tạo nên SIMD width = 16. Tức là PE cluster hiệu dụng không phải chỉ là 12 PE scalar, mà là một cấu trúc dữ liệu 3 × 4 × 16-lane [A1]. Interpretation of PE organization: PE rows = 3 reduction lanes PE cols = 4 output rows in flight SIMD lane = 16 contiguous x positions effective compute tile per beat: 3 (reduction rows) x 4 (output rows) x 16 (output-x lanes) Hình 9-1. Cách hiểu đúng về PE 3×4 khi đi kèm 16 subbank theo chiều width. 9.1. Cửa sổ dữ liệu ngang là phần không được bỏ quên Chỉ có 16 subbank theo width là chưa đủ để chạy conv thật sự. Muốn support conv3×3, maxpool5×5 hoặc DW7×7, subcluster phải có thêm một horizontal window generator. Khối này đọc các vector width liên tiếp từ bank_input rồi tạo các tap ngang K = {1, 3, 5, 7} trước khi đưa vào MAC/comparator. Nếu bỏ qua khối này, kiến trúc mới đúng theo trục height nhưng vẫn thiếu cửa sổ theo width. Bảng 9-1. Các mode cửa sổ ngang cần cóHandbook thiết kế accelerator YOLOv10n descriptor-driven Trang 17 Mode Tap ngang tạo ra Phục vụ primitive K1 x Conv1×1, move, add, detect packaging. K3 x, x+1, x+2 Conv3×3, DW3×3, positional DW3×3. K5 x..x+4 MaxPool5×5 trong SPPF. K7 x..x+6 DW7×7 multi-pass hoặc các biến thể large-kernel. Đây là bổ sung quan trọng so với báo cáo ban đầu. Một accelerator row-oriented muốn đi tới RTL bắt buộc phải có block tạo cửa sổ ngang hoặc buffer tương đương. 9.2. Các mode tính toán của PE Bảng 9-2. Chế độ hoạt động đề xuất của PE_CLUSTER PE_MO DE Ý nghĩa Khai thác 3 PE row như thế nào RS_DEN SE_3x3 Conv 3×3 chuẩn 3 PE row map vào 3 kernel rows. OS_1x1 Conv 1×1 / projection 3 PE row map vào 3 slice reduction theo Cin. DW_3x3 Depthwise 3×3 3 PE row vẫn map vào 3 kernel rows nhưng theo group/channel cục bộ. DW_7x7 Depthwise 7×7 multi-pass 3 PE row xử lý 3 kernel rows mỗi pass. MAXPO OL_5x5 Pooling max PE datapath chuyển từ MAC sang

comparator tree. GEMM Matmul cho PSA PE coi dữ liệu như matrix tile và accumulate theo K dimension. PASS Move/bypass/debug PE không MAC, chỉ chuyển dữ liệu qua pipeline nếu cần. Điểm hay của cách tổ chức này là phần lớn phần cứng datapath có thể chia sẻ. MAC array, local accumulator file, reduction tree theo cột, register pipeline và window generator đều có thể dùng chung. Khác biệt giữa các mode chủ yếu nằm ở cách cấp dữ liệu, cách chọn tap, cách giải nghĩa bank_weight, và kiểu post-process của output. Thực tiễn cho silicon V1: chỉ cần tối ưu cực mạnh cho RS_DENSE_3x3, OS_1x1 và DW_3x3. MAXPOOL_5x5 và GEMM_ATTN có thể chậm hơn đôi chút mà vẫn chấp nhận được vì tần suất xuất hiện thấp hơn và tensor nhỏ hơn. 9.3. Dataflow minh họa cho conv3×3 stride 1 và stride 2 conv3x3, stride 1 PE col0 -> output row 0 uses input rows 0,1,2 PE col1 -> output row 1 uses input rows 1,2,3 PE col2 -> output row 2 uses input rows 2,3,4 PE col3 -> output row 3 uses input rows 3,4,5 resident rows: bank_input_0 : 0,3 bank_input_1 : 1,4 bank_input_2 : 2,5 Hình 9-2. Conv3×3 stride 1 khớp chính xác với quy tắc h mod 3.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 18 conv3x3, stride 2 PE col0 -> output row 0 uses input rows 0,1,2 PE col1 -> output row 2 uses input rows 2,3,4 PE col2 -> output row 4 uses input rows 4,5,6 PE col3 -> output row 6 uses input rows 6,7,8 resident rows: bank_input_0 : 0,3,6 bank_input_1 : 1,4,7 bank_input_2 : 2,5,8 Hình 9-3. Conv3×3 stride 2 cần R_need = 9 input rows resident. Hai ví dụ trên giải thích vì sao modulo-3 input banking là một lựa chọn rất mạnh. Nó vừa khớp trực tiếp với conv3×3 stride 1/2, vừa mở rộng tự nhiên cho các kernel lớn hơn mà không cần thiết kế lại bank layout. 10. PPU: post-processing unit cho psum, activation và tensor ops nhẹ PPU là khối nằm giữa compute core và tensor output path. Tên 'post-processing' ở đây không nên hiểu là hậu xử lý detect kiểu NMS, mà là hậu xử lý cho kết quả tích chập và tensor cục bộ: bias add, requant, activation, elementwise add, pack/unpack, detect packaging và một số biến đổi layout nhẹ. Bảng 10-1. Hai lớp chức năng của PPU Lớp chức năng Các phép xử lý Compute-post path bias add, shift/scale requant, clamp, activation (SiLU/Identity), partial residual add. Tensor-post path upsample duplication, concat offset, move/pack, detect head packaging, swizzle assist. Với YOLOv10n, PPU cần chú ý nhất đến ba điểm. Thứ nhất là activation path cho conv output. Thứ hai là EWISE_ADD để hỗ trợ residual. Thứ ba là packaging path cho detect head, vì inference của YOLOv10 dùng one-to-one head và không cần NMS engine [R4], [R5]. Như vậy PPU không cần phình quá lớn nhưng phải sạch về interface và có thể pipeline. Thiết kế gọn nhất là để PPU nằm sau RPS/bank_output read path: dữ liệu từ PE hoặc từ bank_output có thể đi qua PPU trước khi ghi ACT mode hoặc trước khi được chuyển tới swizzle engine. 11. Descriptor-driven execution model Đây là phần quyết định toàn bộ chiến lược hiện thực. Accelerator trong handbook này vận hành theo mô hình descriptor-driven. CPU/compiler tạo descriptor; controller fetch descriptor; shadow register trong từng super-cluster được nạp; local FSM chạy primitive tương ứng; dữ liệu được nạp, tính, post-process và swizzle/store dựa trên các bound đã tiền

tính.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 19 Compile time on CPU: model graph -> primitive IR -> tensor arena allocation -> layer partition -> tile search -> router profile generation -> post profile generation -> descriptor emission Run time on IP: fetch descriptor -> decode -> load shadow regs -> prefill -> compute -> post / swizzle / store -> rotate roles -> next tile / next layer Hình 11-1. Hai pha rõ ràng: compile time trên CPU và execute time trên IP. 11.1. CPU/compiler phải làm gì 1. Nạp graph YOLOv10n và suy diễn shape của toàn bộ tensor. 2. Hạ graph xuống primitive phần cứng: RS_DENSE_3x3, OS_1x1, DW_3x3, MAXPOOL_5x5, GEMM_ATTN, MOVE, CONCAT, UPSAMPLE, EWISE_ADD, detect packaging. 3. Fold BatchNorm, bias, scale, zero-point và mọi hằng số có thể gộp vào weight hoặc post-profile. 4. Phân loại layer để chọn partition mode ở top-level và local-level. 5. Giải bài toán tile dựa trên Cap_in, Cap_w, Cap_out và nguyên tắc ưu tiên width rồi mới đến channel. 6. Xác định tensor arena và các offset src/dst/skip/weight cho từng tile. 7. Sinh ROUTER_PROFILE và POST_PROFILE. 8. Sinh LAYER_DESC, TILE_DESC và bảng dependency/barrier giữa các nhánh của graph. Kết quả của compiler không phải là một danh sách lệnh từng chu kỳ mà là một mô tả chặt chẽ ở mức tile/layer. Nhờ đó phần cứng vẫn giữ đặc tính chuyên dụng, còn phần mềm gánh trách nhiệm 'tìm mapping tốt nhất'. 11.2. IP phải làm gì 1. Đọc NET_DESC để biết vị trí bảng LAYER_DESC và các tensor arena chính. 2. Fetch LAYER_DESC, cấu hình shadow register và chọn template FSM tương ứng với PE_MODE. 3. Fetch TILE_DESC của layer hiện tại, nạp địa chỉ và bound cục bộ, kích hoạt prefill input/weight. 4. Theo dõi ready/valid của input, weight, router, output bank và fabric lân cận. 5. Khi đủ điều kiện, chạy RUN_COMPUTE; nếu còn reduction pass thì ACCUMULATE_OR_POST; nếu xong thì SWIZZLE_OR_STORE và ROTATE_ROLE. 6. Báo done/error/counter để controller system quyết định qua tile hoặc qua layer. Như vậy IP chỉ còn là một executor của kế hoạch đã tính sẵn. Đây đúng với mục tiêu bạn đặt ra: controller chủ yếu dùng counter và comparator, không cần các block tính toán ràng buộc nặng. 11.3. Vì sao descriptor phải ở mức layer và tileHandbook thiết kế accelerator YOLOv10n descriptor-driven Trang 20 Một descriptor duy nhất cho cả net là quá thô; còn một lệnh cho từng chu kỳ là quá vụn và làm IP giống processor. Mức phù hợp nhất là hai tầng: LAYER_DESC mang ngữ nghĩa chung của layer; TILE_DESC mang địa chỉ, biên tile, role và override cụ thể. ROUTER_PROFILE và POST_PROFILE được tách riêng để tái sử dụng. Cấu trúc này đủ chặt để xác định src/dst rõ ràng mà không làm instruction RAM phình quá nhanh. Bảng 11-1. Phân vai giữa các descriptor Descriptor Vai trò NET_DESC Điểm vào của graph, base pointer của arena, số lượng layer, version/capability. LAYER_DESC Miêu tả primitive, shape, tile cơ sở, mode, partition, pass count, profile_id và con trỏ bảng tile. TILE_DESC Miêu tả tile cụ thể: offset src/dst, valid_w/h, halo, role, cluster mask, override cục bộ. ROUTER_PROFILE Cấu hình đường đi dữ liệu cho một primitive hay một nhóm tile. POST_PROFILE Bias/requant/activation/concat/upsample/detect packaging. BARRIER_DESC hoặc scoreboard table Quan hệ phụ thuộc giữa các nhánh

skip/concat/head. 12. Định dạng nhị phân của descriptor Phần này chốt chính xác format nhị phân khuyến nghị. Các format dưới đây được thiết kế để dễ DMA, dễ align và dễ debug. Một nguyên tắc nhất quán là dùng bội số của 32 bit cho mọi trường lớn, còn các mode/flag dùng bitfield ở những word riêng. LAYER_DESC và TILE_DESC đều chọn kích thước 64 byte để dễ align cache line và dễ truy xuất bằng burst. 12.1. NET_DESC (64 byte) Bảng 12-1. Đề xuất format NET_DESC Word Bits Field Ý nghĩa W0 31:16 MAGIC Ví dụ 0x594F ('YO'). W0 15:8 VERSION Version format descriptor. W0 7:0 FLAGS Cờ tổng quát. W1 31:16 NUM_LAYER Số layer descriptor. W1 15:0 RESERVED Để trống. W2 31:0 LAYER_TABLE_LO Lower 32b của base pointer bảng LAYER_DESC. W3 31:0 LAYER_TABLE_HI Upper 32b của base pointer bảng LAYER_DESC. W4 31:0 WEIGHT_ARENA_LO Base pointer vùng weight. W5 31:0 WEIGHT_ARENA_HI Upper pointer vùng weight. W6 31:0 ACT0_ARENA_LO Base pointer vùng activation A. W7 31:0 ACT0_ARENA_HI Upper pointer vùng activation A. W8 31:0 ACT1_ARENA_LO Base pointer vùng activation B. W9 31:0 ACT1_ARENA_HI Upper pointer vùng activation B. W10 31:0 AUX_ARENA_LO Base pointer scratch/aux. W11 31:0 AUX_ARENA_HI Upper pointer scratch/aux. W12-W15 31:0 RESERVED Dành cho mở rộng sau này. 12.2. LAYER_DESC (64 byte)Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 21 Bảng 12-2. Đề xuất format LAYER_DESC Word Bits Field Ý nghĩa W0 31:24 DESC_TYPE Loại descriptor; ví dụ 0x10 cho layer. W0 23:16 TEMPLATE_ID Mã primitive: RS3, OS1, DW3, MP5, GEMM... W0 15:0 LAYER_ID ID layer trong graph. W1 31:16 CIN_TOTAL Tổng số input channel. W1 15:0 COUT_TOTAL Tổng số output channel. W2 31:16 HIN Chiều cao input. W2 15:0 WIN Chiều rộng input. W3 31:16 HOUT Chiều cao output. W3 15:0 WOUT Chiều rộng output. W4 31:28 KH Kernel height. W4 27:24 KW Kernel width. W4 23:20 SH Stride height. W4 19:16 SW Stride width. W4 15:12 DH Dilation height. W4 11:8 DW Dilation width. W4 7:0 PAD_MODE Mã pad mode hoặc flag pad. W5 31:24 PAD_TOP Padding top. W5 23:16 PAD_BOTTOM Padding bottom. W5 15:8 PAD_LEFT Padding left. W5 7:0 PAD_RIGHT Padding right. W6 31:16 TILE_CIN Input channels cho một tile cục bộ. W6 15:0 TILE_COUT Output channels cho một tile cục bộ. W7 31:16 TILE_W Độ rộng tile theo pixel. W7 15:0 NUM_TILE_HW Số tile spatial của layer. W8 31:28 TOP_PART Partition top-level. W8 27:24 LOCAL_PART Partition local giữa 4 subcluster. W8 23:16 PE_MODE Mode của PE. W8 15:8 ACT_MODE Kiểu activation. W8 7:0 QUANT_MODE Kiểu requant/pack. W9 31:24 R_NEED Resident rows cần có. W9 23:16 R_NEW Số row mới cần nạp mỗi vòng. W9 15:8 Q_IN Số row-slot mỗi bank_input. W9 7:0 Q_OUT Số out-slot mỗi bank_output. W10 31:16 NUM_CIN_PASS Số reduction pass theo Cin. W10 15:0 NUM_K_PASS Số pass theo kernel (ví dụ DW7×7). W11 31:16 ROUTER_PROFILE_ID Chọn ROUTER_PROFILE. W11 15:0 POST_PROFILE_ID Chọn POST_PROFILE. W12 31:24 SRC_IN_TID Tensor ID input. W12 23:16 SRC_W_TID Tensor ID weight. W12 15:8 SRC_SKIP_TID Tensor ID skip/aux. W12 7:0 DST_TID Tensor ID output.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 22 Word Bits Field Ý nghĩa W13 31:0 TILE_TABLE_OFF_LO Offset tới bảng TILE_DESC. W14

31:0 TILE_TABLE_OFF_HI Upper offset tới bảng TILE_DESC. W15 31:0 LAYER_FLAGS Các cờ như keep_on_chip, need_barrier, edge_sensitive... 12.3. TILE_DESC (64 byte) Bảng 12-3. Đề xuất format TILE_DESC Word Bits Field Ý nghĩa W0 31:24 DESC_TYPE Loại descriptor; ví dụ 0x20 cho tile. W0 23:16 FLAGS_HI Cờ tile mức cao. W0 15:8 SC_MASK Mask chọn super-cluster/subcluster đích. W0 7:0 FLAGS_LO Cờ tile mức thấp. W1 31:16 TILE_ID ID tile trong layer. W1 15:0 LAYER_ID Để check nhất quán. W2 31:16 H_OUT0 Output row bắt đầu của tile. W2 15:0 WBLK0 Block width bắt đầu của tile. W3 31:16 CIN0 Input-channel bắt đầu. W3 15:0 COUT0 Output-channel bắt đầu. W4 31:16 VALID_H Chiều cao valid của tile. W4 15:0 VALID_W Chiều rộng valid của tile. W5 31:24 HALO_TOP Số row halo phía trên. W5 23:16 HALO_BOTTOM Số row halo phía dưới. W5 15:8 HALO_LEFT Số cột halo trái. W5 7:0 HALO_RIGHT Số cột halo phải. W6 31:0 SRC_IN_OFF Offset byte của input tile trong tensor arena. W7 31:0 SRC_W_OFF Offset byte của weight tile. W8 31:0 SRC_SKIP_OFF Offset byte của skip/aux tile. W9 31:0 DST_OFF Offset byte đích của output tile. W10 31:16 IN_BASE_H Logical base_h dùng cho modulo-3 banking. W10 15:0 IN_BASE_C Logical base_c của tile. W11 31:16 OUT_BASE_H Logical output base_h. W11 15:0 OUT_BASE_C Logical output base_c. W12 31:16 FIRST_CIN_PASS Pass Cin bắt đầu cho tile này. W12 15:0 NUM_CIN_PASS Số pass Cin cục bộ. W13 31:16 FIRST_K_PASS Pass kernel bắt đầu. W13 15:0 NUM_K_PASS Số pass kernel cục bộ. W14 31:16 ROUTER_OVERRIDE_ID Override profile nếu cần. W14 15:0 POST_OVERRIDE_ID Override post-profile nếu cần. W15 31:0 EXPECTED_CYCLE Ước lượng cycle để perf/debug hoặc watchdog. 12.4. Cờ khuyến nghị cho TILE_DESC Bảng 12-4. Bitfield cờ tile khuyến nghịHandbook thiết kế accelerator YOLOv10n descriptor-driven Trang 23 Bit Tên cờ Ý nghĩa 0 FIRST_TILE Tile đầu của layer. 1 LAST_TILE Tile cuối của layer. 2 EDGE_TILE Tile biên cần valid-mask/padding. 3 HAS_SKIP Tile có đường skip/aux. 4 NEED_SWIZZLE Kết quả cần swizzle sang bank_input kế tiếp. 5 NEED_SPILL Kết quả phải spill ra external arena. 6 BARRIER_BEFORE Phải chờ dependency trước khi chạy. 7 BARRIER_AFTER Phải báo barrier sau khi xong. 8 PREFILL_ONLY Tile chỉ dùng để prefill/resident row. 9 STORE_ONLY Tile chỉ phục vụ writeback/drain. 10 HOLD_SKIP_ROLE Tile được gán cho subcluster đang giữ skip. 11 ROUTER_OVERRIDE_VALID Dùng ROUTER_OVERRIDE_ID. 12 POST_OVERRIDE_VALID Dùng POST_OVERRIDE_ID. 13-31 RESERVED Dành cho mở rộng. 12.5. ROUTER_PROFILE và POST_PROFILE Bảng 12-5. Đề xuất ROUTER_PROFILE 16 byte Word Field chính Ý nghĩa W0 RIN0/1/2 source + dst mask Định nghĩa activation đi từ đâu tới đâu. W1 RIN broadcast mode + RWTx source Cấu hình pattern multicast. W2 RWT horizontal multicast + RPS dst Cấu hình weight và psum route. W3 RPS accumulate mode + swizzle route Bật reduce dọc và feed sang swizzle. Bảng 12-6. Đề xuất POST_PROFILE 16 byte Word Field chính Ý nghĩa W0 bias_en, quant_mode, act_mode, ewise_mode Điều khiển hậu xử lý số học. W1 bias/scale table offset Con trỏ tới bias/scale đã pack sẵn. W2 concat/upsample/detect flags Điều khiển tensor-post path. W3 reserved Dành cho mở rộng. Giữ ROUTER_PROFILE

và POST_PROFILE nhỏ là rất quan trọng. Hai profile này nên được cache/shadow tại super-cluster để dùng lại cho nhiều tile cùng primitive. 13. Đặc tả thanh ghi (register map) cho Controller System và các cluster Phần này mô tả MMIO/CSR ở mức chip và các shadow register nội bộ. Mục tiêu là giúp driver phần mềm, descriptor decoder và RTL có một bản đồ thanh ghi nhất quán. Không nhất thiết mọi shadow register phải lộ hết ra MMIO; nhiều trường có thể chỉ được load nội bộ từ descriptor. Nhưng ít nhất hệ thống cần một tập CSR đủ để khởi động, debug, quan sát trạng thái và inject test. 13.1. Global CSR ở mức chipHandbook thiết kế accelerator YOLOv10n descriptor-driven Trang 24 Bảng 13-1. Global CSR Offset Tên Access Mô tả 0x0000 CTRL RW [0] start, [1] soft_reset, [2] irq_clr, [3] halt_on_err 0x0004 STATUS RO [0] busy, [1] done, [2] irq, [3] error, [7:4] state 0x0008 VERSION RO Version phần cứng và capability bit. 0x000C CAP0 RO num_supercluster, subcluster_per_sc, pe_rows, pe_cols, vec_width 0x0010 NET_DESC_L O RW Lower 32b của base pointer NET_DESC. 0x0014 NET_DESC_H I RW Upper 32b của base pointer NET_DESC. 0x0018 LAYER_STAR T RW Layer bắt đầu cần chạy. 0x001C LAYER_END RW Layer kết thúc cần chạy. 0x0020 IRQ_MASK RW Mask cho done/error/perf threshold. 0x0024 ERR_CODE RO Mã lỗi tổng quát. 0x0028 ERR_INFO0 RO Thông tin lỗi phụ. 0x002C ERR_INFO1 RO Thông tin lỗi phụ. 0x0030 PERF_CTRL RW Enable/reset performance counters. 0x0034 PERF_CYCLE _LO RO Cycle counter toàn chip. 0x0038 PERF_CYCLE _HI RO Upper cycle counter. 0x003C PERF_TILE_D ONE RO Số tile đã hoàn tất. 13.2. CSR của descriptor fetch và barrier manager Bảng 13-2. Descriptor fetch và barrier manager Offset Tên Mô tả 0x0100 DESC_FETCH_CTRL Bật/tắt fetch engine, chế độ single-step. 0x0104 DESC_HEAD_PTR_LO Con trỏ descriptor hiện tại. 0x0108 DESC_HEAD_PTR_HI Upper pointer descriptor hiện tại. 0x010C DESC_STATUS Empty/full/error/checksum status. 0x0120 BARRIER_MASK Bitmap dependency đang chờ. 0x0124 BARRIER_DONE Bitmap dependency đã hoàn tất. 0x0128 SCHED_STATUS Trạng thái scheduler mức chip. 0x012C ACTIVE_LAYER_ID Layer đang hoạt động. 13.3. CSR của mỗi super-cluster Bảng 13-3. CSR của mỗi super-cluster (base = 0x1000 + sc_id*0x1000) Offset Tên Mô tả +0x000 SC_CTRL Enable/disable cluster, single-step, flush local FIFO. +0x004 SC_STATUS busy/idle/error/current role map. +0x008 SC_ROLE_MAP Subcluster0..3 role = RUN/FILL/DRAIN/HOLD. +0x00C SC_ACTIVE_TILE Tile ID đang chạy.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 25 Offset Tên Mô tả +0x010 SC_IN_BEAT_CNT Số beat input đã nạp. +0x014 SC_OUT_BEAT_C NT Số beat output đã ghi/spill. +0x018 SC_ROUTER_STAL L Counter stall do router. +0x01C SC_DMA_STALL Counter stall do external port. 13.4. Shadow register nội bộ của subcluster Ngoài CSR lộ ra MMIO, mỗi subcluster cần một bank shadow register được nạp trực tiếp từ descriptor. Đây mới là phần rất quan trọng cho RTL. Tôi khuyến nghị chia shadow register thành 6 nhóm: layer-level, input-bank, weight-bank, output-bank, router và scheduler local. Cách chia này nối rất tự nhiên với báo cáo trước [A1]. Bảng 13-4. Nhóm shadow register nội bộ của

subcluster Nhóm Trường cốt lõi Layer-l evel LAYER_TYPE, KERNEL_H/W, STRIDE_H/W, PAD_*, CIN_TOTAL, COUT_TOTAL, HIN/WIN/HOUT/WOUT, TILE_CIN/COUT/W, PARTITION_MODE, PE_MODE, ACT_MODE, QUANT_MODE. Input-b ank IN_BASE_ADDR, IN_WBLK, IN_CTILE, IN_QSLOT, IN_BASE_H, IN_BASE_C, IN_ROW_MOD=3. Weightbank W_BASE_ADDR, W_MODE, W_CIN_TILE, W_COUT_TILE, W_PASS_ID, W_NUM_PASS. Outputbank OUT_BASE_ADDR, OUT_WBLK, OUT_COUT_TILE, OUT_QSLOT, OUT_BASE_H, OUT_MODE. Router RINx_SRC_SEL, RINx_DST_MASK, RIN_BCAST_MODE, RWTx_SRC_SEL, RWT_HMCAST_EN, RPSx_DST_SEL, RPS_ACC_MODE. Schedu ler local SC_ROLE, ACTIVE_SC, NEXT_FILL_SC, NEXT_DRAIN_SC, SKIP_HOLD_SC, pass counter, tile counter. Trong RTL, shadow register nên được chụp ở ranh giới LOAD_CFG của tile/layer. Sau đó local datapath chỉ dùng shadow register, không đọc trực tiếp descriptor RAM nữa. Điều này làm timing và verify đơn giản hơn. 13.5. Gợi ý memory map MMIO tổng thể 0x0000 - 0x00FF : Global control/status 0x0100 - 0x01FF : Descriptor fetch / barrier / scheduler 0x0200 - 0x02FF : Performance / debug / error inject 0x1000 - 0x1FFF : Super-cluster 0 0x2000 - 0x2FFF : Super-cluster 1 0x3000 - 0x3FFF : Super-cluster 2 0x4000 - 0x4FFF : Super-cluster 3 0x5000 - 0x5FFF : Reserved for bring-up / scan / BIST Hình 13-1. Memory map MMIO khuyến nghị. 14. Layout dữ liệu, địa chỉ src/dst và cơ chế swizzle Câu hỏi trung tâm bạn đặt ra là: 'lúc này ta xác định rõ luôn địa chỉ src cần load và destination cần ghi dữ liệu cho các layer khác nhau hay sao?' Câu trả lời là có, nhưng ở mức tile descriptor chứ không phải ở mức từng phần tử. Compiler phải xác định rõ base pointer và offset cho input/weight/skip/output của từng tile, đồng thời phát đủ stride-pattern để hardware chỉ cần tăng counter.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 26 14.1. Ba lớp địa chỉ 1. Base pointer toàn cục: chỉ tới arena của input, output, weight, aux hoặc descriptor table. 2. Offset mức layer/tile: chỉ tới tile cụ thể bên trong arena, ví dụ SRC_IN_OFF, SRC_W_OFF, DST_OFF. 3. Counter cục bộ: xb, lane, row_slot, pass_id, pe_col, oc_local, ic_local. Đây là phần runtime tạo ra địa chỉ beat cụ thể. Cách phân lớp này làm giảm đáng kể kích thước descriptor. Thay vì liệt kê từng beat, descriptor chỉ cần chỉ ra điểm bắt đầu và cách tăng. Các trường hợp irregular như concat, upsample hay tile biên được mô tả bằng flag và một vài offset/halo bổ sung. 14.2. Địa chỉ input Given: x = xb * 16 + lane ib = h mod 3 slot = floor(h / 3) mod Q_in Wblk = ceil(Wtile / 16) Address: addr_in = ((slot * Ctile) + c) * Wblk + xb Hình 14-1. Công thức địa chỉ input ở bank_input. Compiler phải tính trước Wblk, Ctile, Q_in và các bound valid_w/h. Runtime chỉ còn việc đọc ib, slot và xb từ các counter. Nếu tile biên không đủ width chia hết cho 16, lane ngoài phạm vi sẽ được mask và ghi PAD_VALUE hoặc bỏ qua read tùy mode. 14.3. Địa chỉ weight Dense conv3x3: addr_w = ((((oc * Cin_tile) + ic) * Kh) + kr) * Kw + kc Conv1x1 (IC-slice over PE rows): slice_id = pe_row addr_w = (oc * Cin_tile) + ic_slice_base + ic_local Depthwise: addr_dw = (((c * Kh) + kr) * Kw + kc) Hình 14-2. Công thức gợi ý cho địa chỉ weight. Các công thức trên không bắt buộc phải đi thẳng vào RTL như phép nhân tổng quát. Compiler có thể precompute stride

constants và truyền xuống dưới dạng addend. Ví dụ tại runtime, thay vì nhân với Kw hay Cin_tile, controller chỉ tăng pointer theo stride_w_const, stride_c_const và stride_oc_const. 14.4. Địa chỉ output h_out = h_base + pe_col * stride_h + tile_iter * (4 * stride_h) obank = pe_col oslot = floor((h_out - h_base) / (4 * stride_h)) mod Q_out addr_o = ((oslot * Cout_tile) + oc) * Wblk_out + xb Hình 14-3. Công thức địa chỉ output/psum ở bank_output. Ở đây compiler phải biết rõ output hiện được giữ ở PSUM mode hay ACT mode. Nếu layer còn pass Cin hoặc pass kernel, out_mode = PSUM và write width phải khớp psum width. Nếu là output cuối, PPU sẽ requant rồi out_mode = ACT. 14.5. Swizzle output inputHandbook thiết kế accelerator YOLOv10n descriptor-driven Trang 27 Swizzle engine là khối bắt buộc nếu muốn nhiều layer chạy nối tiếp on-chip. Lý do là layout của bank_output không giống layout của bank_input. bank_output được chia theo 4 stream output-row, còn bank_input mong muốn modulo-3 row banking và 16-lane theo width. Swizzle giải bài toán này bằng cách giải mã tọa độ tensor logic rồi mã hóa lại sang layout input của layer sau. for each output element (h_out, x, c_out): ib_next = h_out mod 3 slot_next = floor(h_out / 3) mod Q_in_next lane = x mod 16 xb = floor(x / 16) addr_next = ((slot_next * Ctile_next) + c_out) * Wblk_next + xb bank_input_next[ib_next].subbank[lane][addr_next] = output_value Hình 14-4. Công thức swizzle từ bank_output sang bank_input cho layer kế tiếp. Từ đây có thể thấy vì sao concat, upsample, move tensor và residual add đều nên dựa trên cùng một hạ tầng swizzle/layout engine. Chúng không cần MAC, nhưng lại cần đọc tensor theo một format và ghi lại theo format khác. 14.6. Concat, upsample và residual trên cùng một hạ tầng Bảng 14-1. Cách thực hiện các op tensor-aware Op Cách làm Concat theo channel Giữ nguyên (h, x), offset channel của tensor B = CA rồi ghi A vào [0..CA-1], B vào [CA..CA+CB-1]. Upsample nearest Nhân đôi địa chỉ khi ghi: (h,x) -> (2h,2x), (2h,2x+1), (2h+1,2x), (2h+1,2x+1). Residual add Đọc tensor_main và tensor_skip theo cùng layout logic, cộng trong PPU hoặc router adder, ghi về ACT mode hoặc bank_input kế tiếp. Move tensor Bypass qua router + swizzle mà không qua PE MAC datapath. Điểm mấu chốt là các op tensor-aware phải chia sẻ được một nền tảng layout logic. Nếu mỗi op có một datapath riêng, area tăng mà verify rất khó. 15. Flow compiler / pre-compute step by step Phần này mô tả toàn bộ công việc mà CPU/compiler phải làm trước khi IP bắt đầu chạy. Đây chính là lời giải cho yêu cầu cốt lõi của bạn: biến hệ thống thành một máy thực thi descriptor, nơi mọi tham số quan trọng đều đã được tính sẵn offline. Tôi chia flow thành 11 bước rõ ràng để có thể hiện thực cả compiler lẫn testbench. 1. Đọc graph YOLOv10n và xây IR chuẩn hóa: node, tensor, shape, quant param, dependency. 2. Lower từng node sang primitive phần cứng tương ứng. 3. Fold BN/bias/scale và fuse các nhánh có thể fuse ở inference, ví dụ RepVGGDW trong CIB [R3]. 4. Phân tích live range của tensor để xây tensor arena và tái sử dụng bộ nhớ ngoài. 5. Chọn partition top-level và local-level cho từng primitive. 6. Giải bài toán tile bằng capacity model của GLB_BANK và băng thông port ngoài. 7. Tính các offset src/dst/skip/weight của

từng tile và các tham số halo/valid biên. 8. Sinh ROUTER_PROFILE và POST_PROFILE cho từng loại primitive hoặc từng stage. 9. Sinh LAYER_DESC và TILE_DESC, gắn dependency/barrier cho nhánh skip/concat.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 28 10. Pack weight theo thứ tự tiêu thụ của hardware để giảm scatter read. 11. Ghi tất cả descriptor/tensor/weight vào arena và phát con trỏ NET_DESC cho driver/IP. 15.1. Chọn partition như thế nào Tại mỗi primitive, compiler phải trả lời hai câu hỏi: chia việc giữa 4 super-cluster như thế nào, và chia việc giữa 4 subcluster nội bộ như thế nào. Các quyết định này có thể giống hoặc khác nhau. Ví dụ top-level có thể H_SPLIT, còn local-level lại COUT_SPLIT. Bảng 15-1. Quy tắc chọn partition được chốt trong handbook Điều kiện Top-level Local-level Giải thích Depthwise / grouped GROUP_SPLIT hoặc H_SPLIT nhẹ GROUP_SPLIT Mỗi cluster/subcluster xử lý group độc lập; tránh reduce. Pointwise 1×1 / projection COUT_SPLIT COUT_SPLIT Input multicast, output độc lập, không cần reduce. Dense 3×3 resolution cao H_SPLIT + COUT_SPLIT COUT_SPLIT hoặc CIN_SPLIT Giảm resident rows và tránh pressure Cap_in. Cin quá lớn, không đủ Cap_in CIN_SPLIT CIN_SPLIT Chấp nhận reduce psum liên-cluster như phương án bất đắc dĩ. SPPF / MAXPOOL H_SPLIT H_SPLIT hoặc local-only Spatial op hưởng lợi từ locality theo hàng. PSA ở feature map nhỏ Có thể chỉ dùng 1 hoặc 2 super-cluster COUT_SPLIT Ưu tiên đơn giản hóa hơn throughput tuyệt đối. Nói ngắn gọn: GROUP_SPLIT trước, COUT_SPLIT sau, CIN_SPLIT cuối cùng. Đây là quy tắc rất thực dụng cho YOLOv10n và khớp với báo cáo kiến trúc trước [A1]. 15.2. Thuật toán chọn tile Input: primitive, H, W, Cin, Cout, Kh, Kw, stride, Cap_in, Cap_w, Cap_out Output: tile_w, tile_cin, tile_cout, q_in, q_out, num_pass Algorithm: 1. Wpad = align_up(W, 16) 2. compute R_need, R_new, Q_in 3. choose tile_w = W if possible; otherwise reduce to aligned block 4. compute max tile_cin from Cap_in 5. compute max tile_cout from Cap_out and Cap_w 6. choose tile_cin, tile_cout to maximize utilization under bandwidth limit 7. derive num_cin_pass and num_k_pass 8. verify spill / swizzle path availability 9. emit tile grid and per-tile halo/valid fields Hình 15-1. Thuật toán chọn tile theo hướng tối đa hóa width trước. Để tránh runtime phải giải ràng buộc, compiler phải kiểm tra tất cả điều kiện trước. Nếu phát hiện tile không chạy được vì thiếu Cap_in hoặc Cap_out, compiler phải sửa partition hoặc giảm tile. IP không phải nơi xử lý các ngoại lệ quy hoạch này. 15.3. Cách tính src/dst cho từng tile Sau khi có grid tile, compiler gán cho mỗi tile một bộ offset src/dst. Về bản chất, mỗi TILE_DESC mô tả một lát của tensor logic, còn offset là con trỏ tới biểu diễn vật lý trong arena. Cách gán này phụ thuộc việc tensor của layer đang được giữ on-chip, spill ra external arena hay tồn tại dưới dạng skip giữ trong HOLD_SKIP.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 29 tile.src_in_off = tensor_in.base + layout_offset(tile.h0, tile.w0, tile.cin0) tile.src_w_off = weight.base + weight_offset(tile.cin0, tile.cout0, pass) tile.src_skip_off= tensor_skip.base+ layout_offset(...) tile.dst_off = tensor_out.base + layout_offset(tile.h0, tile.w0, tile.cout0) Hình 15-2. Offset trong TILE_DESC là offset tương đối trong arena, không phải địa chỉ từng

phần tử. Điểm mạnh của cách làm này là driver có thể relocate toàn bộ graph bằng cách thay base pointer ở NET_DESC mà không cần sửa từng LAYER_DESC/TILE_DESC. 15.4. Pseudocode compile cho một layer compile_layer(layer): prim = lower_to_primitive(layer) part_top, part_local = choose_partition(prim, shape, caps) tile_cfg = choose_tile(prim, shape, caps, bandwidth) profiles = build_profiles(prim, part_top, part_local) tiles = [] for each spatial tile and channel tile: td = make_tile_desc(layer, prim, tile_cfg, profiles) td.src_in_off = calc_input_offset(...) td.src_w_off = calc_weight_offset(...) td.src_skip_off= calc_skip_offset_if_any(...) td.dst_off = calc_output_offset(...) td.valid_w, td.valid_h, td.halo = calc_edges(...) td.flags = calc_flags(...) tiles.append(td) return LAYER_DESC(..., tile_table_off=emit(tiles), ...) Hình 15-3. Compile một layer thành LAYER_DESC + danh sách TILE_DESC. 16. Runtime controllers và FSM Ở runtime, phần cứng không giải quy hoạch nữa mà chỉ thi hành một FSM hữu hạn. Tuy nhiên FSM phải được tách rõ thành 3 tầng để RTL sạch: tầng chip (controller system), tầng super-cluster (scheduler và arbiter cổng 128b), và tầng subcluster/tile (prefill compute post swizzle/store). 16.1. FSM mức tile IDLE -> LOAD_CFG -> PREFILL_WEIGHT -> PREFILL_INPUT -> WAIT_READY -> RUN_COMPUTE -> ACCUMULATE_OR_POST -> SWIZZLE_OR_STORE -> ROTATE_CLUSTER -> DONE_TILE -> DONE_LAYER Hình 16-1. FSM tile-level khuyến nghị. Bảng 16-1. Ý nghĩa từng trạng thái của FSM tile-level Trạng thái Ý nghĩa chính Điều kiện ra LOAD_CFG Chụp shadow register từ descriptor Descriptor hợp lệ, profile sẵn sàng. PREFILL_WEIG HT Nạp bank_weight hoặc staging FIFO Đủ weight cho pass đầu.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 30 Trạng thái Ý nghĩa chính Điều kiện ra PREFILL_INPU T Nạp đủ resident rows R_need input_ready = 1. WAIT_READY Chờ weight/input/router/fabric đều sẵn sàng all_ready = 1. RUN_COMPUT E PE thực thi một reduction pass pass_done = 1. ACCUMULATE _OR_POST Quyết định còn pass hay sang hậu xử lý last_pass? SWIZZLE_OR_ STORE Ghi output cuối vào bank_input đích hoặc external arena store_done = 1. ROTATE_CLUS TER Cập nhật role RUN/FILL/DRAIN/HOLD role_update_done = 1. DONE_TILE Báo tile hoàn tất next_tile_available? DONE_LAYER Báo layer hoàn tất next_layer_available? Điểm cần để ý là PREFILL_WEIGHT và PREFILL_INPUT có thể overlap một phần nếu external port và weight/input source khác nhau. Tuy nhiên ở V1, để giảm rủi ro, có thể giữ một dependency bảo thủ: phải chắc chắn weight đầu tiên và đủ resident rows đã sẵn sàng trước khi vào WAIT_READY. 16.2. Scheduler 4 pha trong super-cluster At any time: subcluster A = RUNNING subcluster B = FILLING subcluster C = DRAINING subcluster D = HOLD_SKIP After current tile is done: A <- next role B <- next role C <- next role D <- next role Hình 16-2. Vai trò quay vòng của 4 subcluster. Scheduler local phải làm hai việc đồng thời. Thứ nhất là quay vai trò của subcluster. Thứ hai là điều phối external port 128 bit/cycle. Cụ thể, dù super-cluster có 4 subcluster, trong một chu kỳ chỉ một subcluster được 'ăn' beat từ cổng ngoài. Vì vậy FILLING, DRAINING và HOLD_SKIP phải dùng một arbitration policy rõ ràng. Tôi khuyến nghị ưu tiên: RUNNING không dùng cổng ngoài trừ khi

fault; FILLING ưu tiên cao nhất; DRAINING đứng thứ hai; HOLD_SKIP thấp nhất. Bảng 16-2. Luật ưu tiên arbiter của cổng ngoài trong một super-cluster Mức ưu tiên Vai trò Lý do 1 FILLING Nếu tile kế tiếp không prefill kịp, compute sẽ stall. 2 DRAINING Nếu không drain kịp, bank_output sẽ đầy và chặn tile sau. 3 HOLD_SKIP Chỉ cần giữ sẵn sàng skip tensor; thường không gấp bằng fill/drain. 4 RUNNING Lý tưởng là không cần cổng ngoài trong khi chạy. 16.3. Barrier và dependency cho nhánh concat/skip Head của YOLOv10n có các đoạn upsample rồi concat với feature map từ stage khác [R2]. Vì vậy controller system cần một scoreboard hoặc barrier manager. Mỗi TILE_DESC có thể mang cờ BARRIER_BEFORE/BARRIER_AFTER. Khi một nhánh hoàn tất tile của mình, manager set bit done; nhánhHandbook thiết kế accelerator YOLOv10n descriptor-driven Trang 31 concat chỉ được vào RUN_COMPUTE khi tất cả dependency của tile đó đã đủ. if tile.flags.BARRIER_BEFORE: wait until barrier_mask(tile) == barrier_done(tile) run tile if tile.flags.BARRIER_AFTER: barrier_done(tile) |= barrier_emit(tile) Hình 16-3. Barrier đơn giản nhưng đủ cho nhánh concat/skip/head. Barrier manager không cần biết dữ liệu cụ thể là gì. Nó chỉ theo dõi tile readiness và dependency readiness. Điều này giữ control-plane gọn. 17. Ánh xạ layer-by-layer của YOLOv10n lên accelerator Phần này gom lại toàn bộ mapping thành một sổ tay thực dụng cho compiler và cho người viết testbench. Tên layer ở đây bám logic của yolov10n.yaml [R2], còn primitive và mode bám handbook. Bảng 17-1. Mapping layer/stage của YOLOv10n Stage / khối Primitive chính Partition ưu tiên Ghi chú phần cứng Backbone Conv P1/P2/P3 RS_DENSE_3x3 H_SPLIT + COUT_SPLIT Spatial conv resolution cao. C2f ở backbone nông OS_1x1 + RS_DENSE_3x3 + CONCAT COUT_SPLIT 1x1 split/merge, bottleneck 3×3. SCDown OS_1x1 + DW_3x3 GROUP_SPLIT hoặc COUT_SPLIT Điển hình của downsample hiệu quả [R3], [R4]. SPPF OS_1x1 + MAXPOOL_5x5 ×3 + CONCAT + OS_1x1 H_SPLIT Không cần Conv5×5. PSA OS_1x1 + GEMM + DW_3x3 + OS_1x1 COUT_SPLIT Chạy trên feature map nhỏ. C2fCIB OS_1x1 + CIB(DW/1×1[/DW7×7]) + CONCAT + OS_1x1 GROUP/COUT DW7×7 có thể multi-pass sau fuse. Head Upsample + Concat UPSAMPLE + CONCAT + MOVE Theo locality Cần barrier và swizzle. Head fusion conv RS_DENSE_3x3 hoặc OS_1x1 COUT_SPLIT Chạy sau khi concat xong. v10Detect RS_DENSE_3x3 + OS_1x1 + detect packaging COUT_SPLIT One-to-one inference head; không NMS [R5]. Đối với PSA, compiler nên chấp nhận một chiến lược hơi bảo thủ. Vì PSA xuất hiện ở stage sâu với feature map nhỏ trong yolov10n.yaml [R2], hiệu quả tổng hệ thống vẫn chủ yếu do conv3×3, 1×1, SCDown, SPPF, concat và upsample quyết định. Do đó V1 nên tối ưu thật mạnh nhóm primitive này trước. Nếu cần một tiêu chí ưu tiên cho V1: 80% effort cho RS_DENSE_3x3, OS_1x1, DW_3x3, GLB/swizzle/scheduler; 20% effort cho PPU, MAXPOOL_5x5 và GEMM_ATTN tối thiểu. 18. Hai ví dụ worked-example để kiểm tra spec 18.1. Ví dụ A: Conv3×3 stride 2 ở backboneHandbook thiết kế accelerator YOLOv10n descriptor-driven Trang 32 Giả sử một layer backbone Conv3×3 stride 2 với tensor input lớn. Compiler chọn H_SPLIT ở mức super-cluster và COUT_SPLIT ở mức local. Với PE cols =

4, stride_h = 2, compiler tính R_need = 9 và Q_in = 3. Tile descriptor đầu tiên của mỗi subcluster mang IN_BASE_H tương ứng, VALID_W theo biên tile và src/dst offset riêng. 1. Controller fetch LAYER_DESC, nhận ra primitive = RS_DENSE_3x3. 2. Shadow register nhận KH=3, KW=3, SH=SW=2, TILE_W, TILE_CIN, TILE_COUT, Q_IN=3, Q_OUT tương ứng. 3. PREFILL_INPUT nạp đủ 9 resident rows, phân vào bank_input_0/1/2 theo modulo-3. 4. RWT broadcast 3 hàng kernel row tới 3 PE row; RIN đọc 3 hàng input tương ứng cho từng cột PE. 5. PE col0..3 sinh 4 output-row cách nhau 2 dòng; bank_output_0..3 nhận 4 stream output. 6. Nếu còn pass Cin, OUT_MODE = PSUM; nếu pass cuối, qua PPU rồi ACT mode. 7. SWIZZLE_OR_STORE hoặc ghi về arena output, hoặc chuyển trực tiếp sang bank_input của layer kế tiếp nếu layout tương thích. Expected resident rows for first micro-tile: bank_input_0 : rows 0, 3, 6 bank_input_1 : rows 1, 4, 7 bank_input_2 : rows 2, 5, 8 After one 4-row output micro-tile: R_new = 8 rows of input must be streamed for the next window progression Hình 18-1. Conv3×3 stride 2 là test vàng để kiểm tra modulo-3 banking và Q_in = 3. 18.2. Ví dụ B: Head Upsample + Concat + fusion conv Đây là ví dụ quan trọng nhất cho hệ thống vì nó kiểm tra barrier, swizzle, HOLD_SKIP và concat. Một nhánh sâu đã tạo ra feature map P5 và cần upsample lên để concat với feature map P4 giữ từ backbone hoặc neck. 1. Một subcluster ở vai trò HOLD_SKIP giữ tensor P4 hoặc spill offset của nó. 2. Nhánh sâu chạy xong tile P5, output được chuyển qua tensor-post path của PPU để upsample nearest. 3. Upsample không đi qua MAC; address duplication tạo bốn điểm ghi cho mỗi phần tử. 4. Swizzle engine mã hóa lại layout của tensor upsample về dạng bank_input mong muốn của primitive concat/fusion tiếp theo. 5. Barrier manager chỉ cho phép tile fusion conv chạy khi cả tensor upsample và tensor skip đều sẵn sàng. 6. Concat theo channel được thực hiện bằng offset channel, không thay đổi chỉ số (h,x). 7. Sau concat, fusion conv được chạy như RS_DENSE_3x3 hoặc OS_1x1 tùy stage. concat(A, B): for each logical (h, x, c): out[h, x, c] = A[h, x, c] out[h, x, c + CA] = B[h, x, c] upsample_nearest(x): y[2h, 2x] = x[h, x] y[2h, 2x+1] = x[h, x] y[2h+1, 2x] = x[h, x] y[2h+1, 2x+1] = x[h, x] Hình 18-2. Concat và upsample có thể đi trên cùng hạ tầng layout/tensor-post.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 33 19. Decomposition RTL: module nào cần viết Nếu mục tiêu là bắt đầu code SystemVerilog một cách có tổ chức, tôi khuyên decomposition RTL theo module như sau. Việc chia module phải đi cùng nguyên tắc 'mỗi module có trách nhiệm rõ' để UVM/testbench có thể mock hoặc kiểm tra độc lập. Bảng 19-1. Bộ module RTL khuyến nghị cho V1 Module Trách nhiệm chính Interface nổi bật controller_syst em MMIO/CSR, start/stop, irq, barrier, global schedule mmio, desc_mem, irq, cluster_cmd/status desc_fetch_en gine Đọc NET_DESC/LAYER_DESC/TILE_DESC desc_req/resp, shadow_load tensor_dma Nạp/xả external beat 128b/cycle axi-like or custom mem port, local beat port supercluster_ wrapper Local arbiter, 4-phase role scheduler ext_port, neighbor fabrics, subcluster_if[4] subcluster_wr apper Glue logic cho GLB/Router/PE/PPU cfg, ext_in/out, neighbor_in/out,

done/status glb_bank bank_input/weight/output + metadata + local arbiter wr/rd ports, swizzle port, perf counters router_cluster RIN/RWT/RPS routing local bank ports, PE ports, neighbor ports window_gen Tạo cửa sổ ngang K1/K3/K5/K7 vector in, taps out pe_cluster MAC/comparator/GEMM modes act taps, weight lanes, psum out ppu bias/requant/activation/ewise/packaging psum in, act out, config profile swizzle_engine outputinput re-layout, concat, upsample assist src tensor port, dst tensor port perf_mon Counter và snapshot event inputs, CSR readout 19.1. Gợi ý interface SystemVerilog ở mức module module controller_system (input logic clk, input logic rst_n, input logic [31:0] mmio_addr, input logic [31:0] mmio_wdata, input logic mmio_we, input logic mmio_re, output logic [31:0] mmio_rdata, output logic irq, // descriptor memory output logic [63:0] desc_addr, output logic desc_req, input logic [255:0] desc_rdata, input logic desc_valid, // cluster commands/status output cluster_cmd_t sc_cmd [4], input cluster_sts_t sc_sts [4]); endmodule Hình 19-1. Interface gợi ý cho controller_system.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 34 module supercluster_wrapper (input logic clk, input logic rst_n, input sc_cfg_t cfg, // external 128b port input logic [127:0] ext_rdata, input logic ext_rvalid, output logic [127:0] ext_wdata, output logic ext_wvalid, // neighbor fabrics input fabric_pkt_t nbr_in [4], output fabric_pkt_t nbr_out[4], // status output sc_status_t status); endmodule Hình 19-2. Interface gợi ý cho supercluster_wrapper. module subcluster_wrapper (input logic clk, input logic rst_n, input sub_cfg_t cfg, input logic [127:0] ext_beat_in, input logic ext_beat_valid, output logic [127:0] ext_beat_out, output logic ext_beat_out_valid, input fabric_pkt_t nbr_in [4], output fabric_pkt_t nbr_out[4], output sub_status_t status); endmodule Hình 19-3. Interface gợi ý cho subcluster_wrapper. Các kiểu dữ liệu cấu trúc như cluster_cmd_t, sc_cfg_t, sub_cfg_t hay fabric_pkt_t nên được định nghĩa trong một gói package chung. Điều này đặc biệt hữu ích cho testbench và tránh việc field của descriptor bị hiểu khác nhau ở các module khác nhau. 19.2. Pipeline nội bộ của subclusterHandbook thiết kế accelerator YOLOv10n descriptor-driven Trang 35 +----------+ ext/fabric ---> | GLB_BANK | <--- swizzle writeback | input/weight/output | +----------+----------+ | v +----------+ | ROUTER_CLUSTER | | RIN / RWT / RPS | +----------+----------+ | v +----------+ | WINDOW_GEN | | K1/K3/K5/K7 taps | +----------+----------+ | v +----------+ | PE_CLUSTER | | MAC / MAX / GEMM | +----------+----------+ | v +----------+ | PPU | +----------+----------+ | v +----------+ | bank_output / | | swizzle / store | +----------+ Hình 19-4. Pipeline logic bên trong subcluster. Dù sơ đồ vẽ tuần tự, các khối này không nhất thiết luôn mắc tuyến tính. ROUTER có thể bypass trực tiếp sang bank_output hoặc swizzle ở các primitive MOVE/CONCAT/UPSAMPLE. 19.3. Những nơi nên thêm assertion từ đầu • Không cho phép RUN_COMPUTE khi weight_ready = 0 hoặc input_ready = 0. • Không cho phép bank_output write vào slot không hợp lệ hoặc slot đang full khi OUT_MODE = PSUM. • Khi NEED_SWIZZLE = 1 thì phải có dst bank_input hợp lệ và swizzle route mở. • Khi

HAS_SKIP = 1 thì BARRIER_BEFORE không được bỏ qua. • Nếu PARTITION = CIN_SPLIT thì RPS_ACC_MODE phải ở chế độ reduce liên-cluster hoặc local accumulate tương đương. • Ở tile biên, lane hoặc row ngoài valid range phải được pad hoặc masked nhất quán. 20. Kế hoạch verify, chỉ số đo và lộ trình bring-up Phần verify nên đi theo thứ tự từ memory correctness đến primitive, rồi mới lên graph YOLOv10n rút gọn. Nếu làm ngược, lỗi layout và lỗi slot mapping sẽ rất khó truy vết. Báo cáo trước cũng đã gợi ý đúng hướng này [A1]; handbook chỉ cụ thể hóa thêm thành plan khả thi hơn. Bảng 20-1. Lộ trình verify khuyến nghịHandbook thiết kế accelerator YOLOv10n descriptor-driven Trang 36 Giai đoạn Mục tiêu Golden reference Phase 1 bank_input modulo-3, 16 subbank, row-slot ring Python/NumPy model của address mapping. Phase 2 PE 3×4, 4-bank output, psum reduce theo cột Python conv kernel model. Phase 3 router local + weight multicast + psum vertical Directed tests với packet tracing. Phase 4 RS_DENSE_3x3, OS_1x1, DW_3x3, MAXPOOL_5x5 Golden primitive models. Phase 5 CONCAT, UPSAMPLE, EWISE_ADD, swizzle Tensor layout checker. Phase 6 PSA mini-case và detect packaging Reduced model / shape-specific reference. Phase 7 Chuỗi layer YOLOv10n rút gọn Pytorch/NumPy end-to-end subgraph compare. 20.1. Bộ testcase tối thiểu phải có 1. Memory write/read input với W = 16, 32, 48 và các trường hợp W không chia hết 16. 2. Conv3×3 stride 1 với H nhỏ để quan sát rõ resident rows 0..5. 3. Conv3×3 stride 2 với resident rows 0..8 và Q_in = 3. 4. Conv1×1 để kiểm tra 3 PE row hoạt động như reduction slice theo channel. 5. Depthwise3×3 với GROUP_SPLIT giữa 4 subcluster. 6. SPPF path: 1×1 -> maxpool5×5 ×3 -> concat -> 1×1. 7. Upsample + Concat + fusion conv để kiểm tra barrier và swizzle. 8. Residual add để kiểm tra elementwise alignment. 9. PSA mini-case trên feature map nhỏ để kiểm tra GEMM_ATTN và positional DW3×3. 20.2. Chỉ số nên đo ngay từ V1 Bảng 20-2. Performance / efficiency counters nên có Counter Ý nghĩa PE utilization Tỷ lệ PE thực sự MAC/MAX so với thời gian active. Input reuse rate Số lần một vector input được tái dùng trước khi bị thay. Weight reuse rate Số output dùng chung một tile weight. DRAM beat per output pixel Chỉ số hiệu quả memory quan trọng nhất. Router stall cycles Thời gian compute bị kẹt bởi routing/fabric. GLB occupancy Mức dùng của input/weight/output banks. Cycle per tile / per layer Độ dài thực thi để so với compiler estimate. Swizzle throughput Xem swizzle có trở thành bottleneck không. Những counter này giúp vòng lặp compiler-hardware khép kín. Khi thấy router stall cao, compiler có thể đổi partition hoặc giảm broadcast xa. Khi thấy DRAM beat per output pixel cao, compiler có thể thay tile hoặc giữ tensor on-chip lâu hơn. 20.3. Lộ trình silicon V1 V2 • V1: RS_DENSE_3x3, OS_1x1, DW_3x3, MAXPOOL_5x5, MOVE/CONCAT/UPSAMPLE, EWISE_ADD, detect packaging, swizzle hoàn chỉnh. • V1+: tối ưu pipeline, weight staging, perf counter, better arbiter, UVM regression.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 37 • V2: DW_7x7 tốt hơn, GEMM_ATTN nhanh hơn, sparse support, fabric thông minh hơn, tile prefetch sâu hơn. • V2+: xem xét compression, zero-skipping, power gating theo primitive,

hoặc precision scaling. 21. Kết luận thực thi Nếu chốt theo handbook này, kiến trúc ban đầu của bạn đã được nâng từ mức 'ý tưởng đúng hướng' lên mức 'spec đủ chặt để code RTL'. Điểm quan trọng nhất là thiết kế không còn phụ thuộc vào suy luận runtime. CPU/compiler giải toàn bộ ràng buộc trước; descriptor mang quyết định xuống IP; phần cứng chỉ thi hành bằng counter, comparator, DMA, router-config và FSM. Các quyết định kỹ thuật cần xem như đã chốt trong phiên bản hiện tại là: 4 super-cluster theo lưới 2×2; mỗi super-cluster có 4 subcluster; mỗi subcluster có GLB_BANK với 3 bank_input modulo-3, 3 bank_weight, 4 bank_output; ROUTER_CLUSTER tách RIN/RWT/RPS; PE_CLUSTER được hiểu là 3×4×16-lane với window generator K1/K3/K5/K7; PPU xử lý bias/requant/activation/tensor-post; và toàn bộ hệ thống chạy descriptor-driven. Về phần mềm, công việc còn lại là viết compiler/driver sinh LAYER_DESC và TILE_DESC đúng theo format đã chốt. Về phần cứng, công việc còn lại là decomposition thành module RTL, viết address generator dựa trên shadow register, hiện thực ring-buffer 4 pha, swizzle engine và các primitive lõi. Nói cách khác, từ handbook này có thể bước tiếp sang bốn deliverable rất cụ thể: (1) package định nghĩa kiểu dữ liệu/descriptor trong SystemVerilog; (2) skeleton RTL cho controller + supercluster + subcluster; (3) golden model Python/NumPy cho address/dataflow; (4) testplan/UVM sequence bám đúng descriptor format. Phụ lục A. Ví dụ descriptor cho một layer Conv3×3 stride 2 Phụ lục này cho một ví dụ minh họa cách điền LAYER_DESC/TILE_DESC. Các giá trị chỉ nhằm minh họa cơ chế và không khóa bạn vào đúng shape này. Giả sử layer có HIN=80, WIN=80, CIN=128, COUT=256, kernel=3×3, stride=2; compiler chọn TILE_W=80, TILE_CIN=32, TILE_COUT=64, PE_MODE=RS_DENSE_3x3, PARTITION_TOP=H_SPLIT, PARTITION_LOCAL=COUT_SPLIT. Bảng A-1. Ví dụ các trường trọng yếu của LAYER_DESC Field Giá trị ví dụ Ghi chú TEMPLATE_ID RS3 Conv3×3 dense. CIN_TOTAL / COUT_TOTAL 128 / 256 Tổng channel của layer. HIN / WIN / HOUT / WOUT 80 / 80 / 40 / 40 Stride 2. KH / KW / SH / SW 3 / 3 / 2 / 2 Kernel và stride. TILE_CIN / TILE_COUT / TILE_W 32 / 64 / 80 Tile giữ full width. R_NEED / Q_IN 9 / 3 Conv3×3 stride 2. NUM_CIN_PASS 4 128 channel / 32 channel mỗi tile. NUM_K_PASS 1 Vì kernel 3×3. ROUTER_PROFILE_ID 0x0003 Profile conv3×3 stride 2. POST_PROFILE_ID 0x0001 Bias + requant + activation.Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 38 Illustrative TILE_DESC for tile 0 ---------- tile_id = 0 h_out0 = 0 wblk0 = 0 cin0 = 0 cout0 = 0 valid_h/valid_w = 4 / 80 halo = top=0 bottom=1 left=0 right=0 src_in_off = 0x00000000 src_w_off = 0x00000000 src_skip_off = 0x00000000 (unused) dst_off = 0x00000000 in_base_h = 0 in_base_c = 0 out_base_h = 0 out_base_c = 0 flags = FIRST_TILE | EDGE_TILE Hình A-1. Ví dụ TILE_DESC minh họa cho tile đầu tiên. Phụ lục B. Thuật ngữ và chữ viết tắt Bảng B-1. Thuật ngữ dùng xuyên suốt handbook Thuật ngữ Ý nghĩa Super-cluster Cụm lớn mà bạn gọi ban đầu là 1 CLUSTER; ở handbook này là một khối có 4 subcluster. Subcluster Đơn vị cục bộ gồm GLB_BANK + ROUTER_CLUSTER + PE_CLUSTER. GLB_BANK Local SRAM subsystem của subcluster. RIN

/ RWT / RPS Router cho activation, weight và psum/output. PPU Post-processing unit cho bias/requant/activation/tensor-post. Swizzle Biến đổi layout output input hoặc layout tensor theo op. RS_DENSE_3x3 Mode conv3×3 row-stationary. OS_1x1 Mode conv1×1 theo hướng output-stationary / input-channel slice. GROUP_SPLIT / COUT_SPLIT / CIN_SPLIT Ba mode partition khuyến nghị. R_need / Q_in / Q_out Số row resident cần giữ, số row-slot input, số out-slot output. References

[A1] Báo cáo nội bộ: “Báo cáo nghiên cứu kiến trúc memory và dataflow cho accelerator AI hướng YOLOv10n”, bản PDF đã tải lên trong hội thoại. Đây là cơ sở nội bộ cho modulo-3 row banking, 4-bank output, swizzle engine, ring scheduler, primitive mapping và lộ trình verify. [R1] THU-MIG. YOLOv10 repository. GitHub. https://github.com/THU-MIG/yolov10 [R2] THU-MIG. yolov10n.yaml, official model definition. https://github.com/THU-MIG/yolov10/blob/main/ultralytics/cfg/models/v10/yolov10n.yaml [R3] THU-MIG. block.py in Ultralytics modules. SCDown, SPPF, PSA, CIB, RepVGGDW, C2fCIB. https://github.com/THU-MIG/yolov10/blob/main/ultralytics/nn/modules/block.py [R4] Ao Wang et al. YOLOv10: Real-Time End-to-End Object Detection. arXiv:2405.14458. https://arxiv.org/abs/2405.14458Handbook thiết kế accelerator YOLOv10n descriptor-driven Trang 39 [R5] Ultralytics Docs. YOLOv10 overview and NMS-free inference notes. https://docs.ultralytics.com/models/yolov10/ [R6] Yu-Hsin Chen, Tushar Krishna, Joel S. Emer, Vivienne Sze. Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks. ISCA 2016. DOI: 10.1145/3007787.3001177 [R7] Yu-Hsin Chen, Tien-Ju Yang, Joel S. Emer, Vivienne Sze. Eyeriss v2: A Flexible Accelerator for Emerging Deep Neural Networks on Mobile Devices. IEEE JETCAS 2019. DOI: 10.1109/JETCAS.2019.2910232 / arXiv:1807.07928